

Algoritmos e Lógica de Programação



AUTORIA
Cleber Jose Semensate Santos

Bem vindo(a)!

SEJA BEM-VINDO(A)!

Computadores evoluíram e, hoje em dia, fazem parte do dia a dia das pessoas. Celulares, computadores, relógios, carros e até eletrodomésticos possuem programas e, para criar programas, é necessário dominar a linguagem de programação.

Esta obra pretende introduzir uma linguagem de programação específica, o Python, com o objetivo de auxiliá-lo a desenvolver seus próprios programas. Serão abordados os conceitos básicos dessa linguagem, de modo que você possa utilizar o mesmo conceito apresentado em outras linguagens de programação.

No primeiro capítulo da obra, apresentaremos os conceitos básicos dos computadores para explicar como os softwares funcionam. Dessa forma, esperamos que você consiga entender a importância da programação. Além disso, esse capítulo explicará como você deve configurar seu computador para desenvolver os programas que serão realizados nos capítulos posteriores.

No capítulo seguinte, introduziremos um conceito básico de linguagem de programação, a variável, que é um espaço de memória onde podemos guardar informação. Nesse mesmo capítulo, observaremos que, com as variáveis, também são realizadas operações.

No terceiro capítulo, o conceito de condições será apresentado. Os programas executam códigos capazes de realizar decisões e modificar sua execução de acordo com as condições em que o programa se encontra.

Já no quarto capítulo, será trabalhado o conceito de laço de repetição ou, em inglês, loops, nos quais partes de código podem ser copiadas, possibilitando a realização de tarefas repetidas.

Uma das características importantes dos programas é interagir com os usuários. Nesse sentido, o quinto capítulo explicará como você pode mostrar mensagens e receber entrada do usuário.

Uma das vantagens de programas e códigos é você poder reutilizá-los com o uso de funções, como mostrado no sétimo capítulo. Mostraremos, assim, os princípios básicos da programação para que você possa desenvolver programas simples.

Boa leitura!

| Unidade 1

Introdução à lógica de programação e ao ambiente de programação Python



AUTORIA

Cleber Jose Semensate Santos

Introdução

Prezado(a) aluno(a),

Este capítulo é uma introdução a algoritmos e programação. Aqui serão abordados os princípios que embasam esse conteúdo.

Para acompanhar, não é necessário ter um conhecimento prévio de programação ou ser um hacker de computadores. Parte-se do preceito de que o leitor sabe somente como interagir com o computador, ligar, desligar, instalar e utilizar programas e acessar a internet.

Sendo assim, pretende-se, com este capítulo, estimular a curiosidade por esta disciplina, introduzindo os conceitos básicos, de forma que o leitor entenda a importância da programação e da linguagem de programação Python. Além disso, pretende-se preparar o ambiente, instalando o programa que utilizaremos, chamado PyCharm.

Resolução de desafios lógicos



AUTORIA
Cleber Jose Semensate Santos

O ser humano, desde os tempos mais remotos, sempre foi fascinado pelo conhecimento do mundo ao seu redor. Além da curiosidade, o ser humano buscou sempre conhecer o funcionamento do mundo como forma essencial para sua sobrevivência. Os ciclos da natureza, as estações do ano, as temporadas de colheitas de vegetais e os ciclos de procriação de animais são alguns dos exemplos de conhecimentos adquiridos pelo homem ao longo de milhares de anos e que mostram o quanto o universo é organizado e regular.

De modo similar, o homem busca sempre soluções por meio da descoberta de padrões, também conhecidos como relações de causalidade, para solucionar os problemas que ocorrem cotidianamente em sua existência. Assim, o conhecimento adquirido sob forma empírica, isto é, a partir da experimentação prática, aos poucos, foi sendo substituído por um conhecimento mais formalizado, relacionado à descrição de fatos ordenados ou às leis de formação, que convencionamos chamar de leis da natureza.

Observe que a natureza não está subordinada a nenhuma lei do homem. Lei, aqui, possui o significado de uma obrigatoriedade, observada pelo homem, de um determinado comportamento ou fato acontecer, mediante algumas condições previamente definidas. Trata-se de uma forma de o homem descrever o comportamento da natureza, de forma a prever a sua regularidade.

Dizer, por exemplo, que uma das leis da natureza é que “o Sol surgirá no horizonte no começo do dia”, consiste apenas em descrever um comportamento regular do nosso astro-rei. Ele um dia poderá explodir e não nascer mais no horizonte, pouco importando a lei da natureza sobre o seu surgimento.

Elaboramos constantemente esse entendimento de tudo o que acontece ao nosso redor por meio de um processo de pensamento chamado **raciocínio**. O uso da razão, ou raciocínio, pode ser descrito de forma livre como a capacidade do ser humano de estabelecer relações de causa e efeito a partir de determinados fatos observados.

Ao observarmos, por exemplo, que uma goiabeira produz goiabas em abundância somente em uma determinada época do ano — digamos, no mês de março —, usamos a razão para concluir que o mês de março é o mês da colheita de goiabas nas goiabeiras. Logo, caso queiramos comer goiabas, vamos ter que esperar o mês de março chegar.



Na Grécia Antiga, em torno do ano de 322 a.C., o filósofo Aristóteles se dedicou a estudar e formalizar esse modo de pensamento, promovendo a descrição e formalização do uso da razão, que, em grego, tem o nome de *logos*, surgindo, então, a **lógica**, uma área de estudos da Filosofia. Na lógica de Aristóteles, o raciocínio pode ser representado por meio de declarações, também chamadas **premissas**, que descrevem fatos ou conhecimentos de senso comum.

A representação de modo ordenado e sequencial dessas premissas descreve o raciocínio a ser utilizado para resolver um problema ou, ainda, “descobrir a verdade”. Como exemplo, considere o conjunto de premissas abaixo:

- a) Duas pessoas são irmãs entre si quando ambas possuem ao menos um mesmo pai ou mãe.
- b) Maria é filha de Pedro.
- c) Beto é filho de Pedro.

Assumindo que Pedro é o mesmo indivíduo nas premissas 2 e 3, o que podemos concluir? Conclui-se que Beto é irmão de Maria ou Maria é irmã de Beto.

A lógica é também aplicada extensivamente na matemática, em que leis e teoremas foram descobertos e formalizados diretamente pelo processo conhecido como **demonstração de teorema**. Considere o exemplo:

$$X = A$$

$$Y = A$$

Logo, $X = Y$.

Essa é a apresentação matemática formal da seguinte premissa: “se duas grandezas são iguais a uma terceira, então são iguais entre si”. Deve-se notar que a representação matemática é totalmente simbólica: pode ser qualquer coisa; ainda assim, a representação matemática anterior permanecerá válida. A lógica é extensivamente empregada em todas as áreas das ciências e o conhecimento científico está formalmente representado segundo a lógica. A aplicação da lógica impõe uma abordagem clara e direta frente ao problema que se deseja resolver, exigindo uma análise de cada parte constituinte.

Operadores lógicos



AUTORIA
Cleber Jose Semensate Santos

Note que tanto o método indutivo quanto o dedutivo são inteiramente suportados pelas premissas, que devem ser claramente apresentadas de forma direta e concisa. Para a construção das premissas, são utilizados **operadores lógicos**, que estabelecem as relações entre os elementos que compõem a premissa, conforme lecionam Russell e Norvig (2013).

Operador de IMPLICAÇÃO ($A \rightarrow B$)

Esse operador é extensivamente utilizado em lógica e constitui a operação lógica **CAUSA** $\rightarrow$ **EFEITO**.

Quando se diz que A implica B ($A \rightarrow B$), isso significa dizer que quando A acontece, ou é verdadeiro (CAUSA), B necessariamente acontece (EFEITO).

Vejamos um exemplo, em que se tem a premissa “sempre que chove, as calçadas da rua ficam molhadas”. Temos a seguinte representação:

$(CHUVA) \rightarrow (CALÇADA\ MOLHADA)$

No exemplo, CHUVA é a causa e o efeito, ou a consequência, são as CALÇADAS MOLHADAS. As premissas podem se relacionar entre si, em uma situação causa $\rightarrow$ efeito, produzindo uma sequência dedutiva lógica, que conduzirá à solução ou à conclusão.

Abaixo, podemos analisar a **tabela verdade do operador IMPLICAÇÃO**, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como fato VERDADEIRO e F como fato FALSO.



Quadro 1 – Tabela verdade do operador IMPLICAÇÃO

A	B	$A \rightarrow B$
F	F	V
F	V	V
V	F	F
V	V	V

Fonte: o autor.

O operador $A \rightarrow B$ somente é falso (o resultado é F) quando **A é verdadeiro e B é falso**.



Operador OU (A + B)

Esse operador é também muito utilizado em lógica e constitui a operação que relaciona dois fatos A e B, que podem ocorrer **conjuntamente ou de forma alternada**. Quando se diz A ou B (A + B), significa dizer que: A acontece e B não acontece; A não acontece e B acontece; A e B acontecem juntos.

Vejamos um exemplo com a premissa “todo animal bovino possui cores BRANCA ou PRETA”. Temos a seguinte representação:

(COR DE BOVINOS): (BRANCA + PRETA)

No exemplo, um BOVINO poderá ser de cor BRANCA, poderá ser de cor PRETA e poderá ser de duas cores, BRANCA e PRETA (bovinos malhados). Isso significa dizer que um bovino obrigatoriamente deve conter ao menos uma dessas duas cores. Qualquer outro animal que não contenha ao menos essas duas cores não será um bovino. Um bovino não poderá ser verde, amarelo, vermelho, enfim, quaisquer outras cores que não sejam as acima citadas.

O operador OU pode ser aplicado para mais de dois elementos. Sabe-se, por exemplo, que existem bovinos de cor marrom e, ainda, bovinos com combinações dessa cor com as outras duas, branca e preta. Assim, podemos atualizar de forma mais correta a premissa acima:

(COR DE BOVINOS): (BRANCA + PRETA + MARROM)

Abaixo, vejamos a **tabela verdade do operador OU**, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como fato VERDADEIRO e F como fato FALSO:



Quadro 2 - Tabela verdade do operador OU

A	B	A + B
F	F	F
F	V	V
V	F	V
V	V	V

Fonte: o autor.

O operador $A + B$ somente é falso (o resultado é F) quando **A é falso e B é falso**.



Operador E (A . B)

Trata-se do operador lógico que relaciona dois fatos, A e B, que podem ocorrer **somente juntos**. Quando se diz A e B (A.B), significa dizer que A acontece (é verdadeiro) e B, necessariamente, também acontece (é verdadeiro).

Vejamos abaixo um exemplo com a premissa “toda autorização de pagamento deve ser assinada pelo presidente e pelo tesoureiro”. Temos a seguinte representação:

(AUTORIZAR PAGAMENTO): (ASSINATURA PRESIDENTE . TESOUREIRO)

No exemplo, o PRESIDENTE e o TESOUREIRO devem, ambos, assinar a autorização. Se somente um deles, isoladamente, assinar o documento, sem a assinatura do outro, não haverá autorização.

O operador E pode ser aplicado para mais de dois elementos. Em uma dada empresa, por exemplo, exige-se que também o vice-presidente assine a autorização. Assim, para essa empresa, podemos atualizar a premissa acima:

(AUTORIZAR PAGAMENTO): (ASSINATURA PRESIDENTE . TESOUREIRO . VICE-PRESIDENTE)

Abaixo, temos a **tabela verdade do operador E**, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como fato VERDADEIRO e F como fato FALSO.



Quadro 3 - Tabela do operador E

A	B	A . B
F	F	F
F	V	F
V	F	F
V	V	V

Fonte: o autor.

O operador A . B somente é verdadeiro (o resultado é V) quando **A é verdadeiro e B é verdadeiro**.



Operador de NEGAÇÃO ($\sim A$)

Esse operador é o mais simples utilizado em lógica e constitui a operação lógica de inversão do estado lógico do elemento da premissa. Quando se diz “não A”, ou a negação de A, NÃO A ($\sim A$) significa dizer que, se A existe ou é verdadeiro, $\sim A$ não existe ou é falso.

Vejamos um exemplo com a premissa “todo ser vivo não pode estar morto” ou, ainda “a morte é a negação da vida”. Temos a seguinte representação:

(MORTE): ($\sim VIDA$)

No exemplo, um ser vivo não pode estar morto e um morto não pode estar vivo. São fatos opostos um em relação ao outro. O operador $\sim$ pode ser aplicado mais de uma vez no mesmo elemento.

Tomando o exemplo anterior, pode-se também enunciar:

(VIDA): $\sim(\sim VIDA)$

Esse enunciado pode ser lido como “a vida é a negação da negação da vida”. Como negação de vida é a morte, então a negação da negação é a vida. Assim, (A): $\sim(\sim A)$, isto é, a negação da negação de A é o próprio A.

Abaixo, temos a **tabela verdade do operador NEGAÇÃO**, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como fato VERDADEIRO e F como fato FALSO.

Quadro 4 – Tabela verdade do operador NEGAÇÃO

A	$\sim A$
F	V
V	F

Fonte: o autor.



Estratégia de desenvolvimento de programas de computadores



AUTORIA
Cleber Jose Semensate Santos

Uma vez que a lógica, seus métodos e seus operadores formalizam o nosso processo de raciocínio, deve-se empregar esse conhecimento para elaborar **programas de computadores**. Computadores, como todos sabemos, são máquinas, ou seja, não têm a capacidade de pensar. Mesmo com todos os atuais recursos computacionais, as formas de comunicação e as capacidades de processamento de dados muitíssimo superiores à capacidade humana, os computadores não conseguem estabelecer seus próprios processos de raciocínio nem conseguem criar por si mesmos qualquer iniciativa de ação.

Até o presente momento, todos os computadores executam uma **sequência de comandos**, conhecida como programa, e tal programa foi anteriormente criado por um ser humano, um programador. Esse ser humano, o **programador**, é quem detém o conhecimento ou, ainda, o uso da razão, para elaborar a sequência lógica que o computador vai executar.

Observe que o programador necessita dominar o conhecimento lógico o suficiente para a elaboração dessa sequência de instruções, também chamada de comandos, que será executada pela máquina. De forma análoga ao processo de dedução lógica, não existe possibilidade de declarações vagas, premissas inválidas ou de sentido incompreensível. Máquinas não são inteligentes.

A **programação de computadores** é o processo de registrar na memória da máquina essa sequência de comandos, escrita em uma **linguagem de programação** de computadores. Existe uma vasta quantidade de diferentes linguagens de programação. Felizmente, não é necessário conhecer todas para programar um computador. É necessário saber **criar um algoritmo**.

Algoritmos

De acordo com Cormen *et al.* (2012), um **algoritmo** é uma sequência de instruções, escritas de forma clara, ordenada e finita e elaborada para realizar uma determinada tarefa ou resolver um problema. Observe o quão próximo é o processo de análise lógica da definição de algoritmo: como a lógica procura descrever, com o uso da razão, o mundo ao seu redor, todo o funcionamento do mundo pode ser “algoritmizável”. Qualquer fato, qualquer cenário, qualquer método ou procedimento pode ser descrito com um algoritmo.

Segundo Cormen (2014), a **criação de algoritmos** obedece à sequência descrita a seguir:

1º passo

DESCRIÇÃO DO PROBLEMA A RESOLVER OU TAREFA A SER FEITA

Essa descrição necessita ser clara e objetiva, definindo-se com precisão qual é a tarefa a ser realizada. Com frequência, o próprio título do algoritmo é suficiente como descrição. Por exemplo, “Receita de pudim de leite condensado” define exatamente o objetivo final.

2º passo

DECOMPOR A TAREFA OU O PROBLEMA EM TODAS AS AÇÕES NECESSÁRIAS PARA A REALIZAÇÃO

Deve-se também relacionar todos os itens necessários para a realização. Nenhuma ação, tecnicamente chamada instrução, pode ser esquecida ou implicitamente entendida.

3º passo

ORDENAÇÃO LÓGICA DAS AÇÕES

Deve-se proceder à ordenação das ações de modo que, ao final, a tarefa ou o problema esteja resolvido com sucesso.

4º passo

CONCLUSÃO

Deve-se avaliar a sequência de instruções, determinando se o objetivo foi alcançado, ou seja, deve-se testar a sequência, executando passo a passo cada instrução na ordem estabelecida e observando se, ao final, a tarefa foi realmente concluída com êxito.

Esse teste é conhecido como **teste de mesa**. Caso a tarefa ou o problema não tenha sido concluído, retorne ao 2º passo, verificando a falta de alguma instrução, e siga novamente o 3º e o 4º passos, sucessivamente, até que o algoritmo esteja certo. Caso a tarefa ou problema tenha sido concluído com êxito, deve-se parar as ações, já que seu algoritmo estará pronto, conforme leciona Cormen *et al.* (2012).

O ambiente de programação Python e as características da linguagem



AUTORIA
Cleber Jose Semensate Santos

O processo de transformar um algoritmo em um programa de computador é conhecido como **codificação**. De fato, os computadores são programados por uma linguagem específica, usualmente denominada linguagem de programação.

Programar, no contexto das ciências da computação, corresponde à transcrição da sequência de instruções do algoritmo em uma sequência de instruções de máquina — chamadas mais tecnicamente de **códigos de máquina** –, gerando-se, assim, um programa que poderá ser executado pelo computador.

Note que essa relação do algoritmo com o programa codificado em linguagem de programação não é direta. Uma instrução do algoritmo pode necessitar de uma sequência de várias instruções na linguagem de programação. O inverso, dependendo da linguagem de programação, também pode ocorrer: uma sequência de várias instruções do algoritmo pode ser codificada por uma única instrução de programação.

Dentre a quantidade realmente grande de linguagens de programação existentes, algumas são mais próximas do modo formal de escrever os algoritmos, tornando fácil o processo de codificação. Uma dessas linguagens é a Python.

Introdução à Linguagem Python

Python é uma linguagem recente no universo da computação. Criada em 1991 por Guido van Rossum, Python é hoje uma das linguagens de programação mais utilizadas no mundo, sendo empregada no desenvolvimento de centenas de milhares de aplicativos por empresas como Microsoft, Google, Amazon, dentre outras.

O grande sucesso da linguagem Python vem da sua facilidade para programação, sendo caracterizada pelo próprio autor como uma linguagem utilizada para drasticamente reduzir o tempo de desenvolvimento de aplicações.

Assim, conforme Maruch e Maruch (2006), Python tem vantagens únicas, como vemos a seguir:

Linguagem clara e simples

Python é muito fácil de entender, ao menos para os programadores de língua inglesa, pois as instruções são muito próximas das utilizadas para a construção do algoritmo. Mesmo quem nunca tenha visto Python antes consegue entender com pouquíssima dificuldade um programa escrito na linguagem, interpretando o seu funcionamento.

Linguagem multiparadigma

Python aceita tanto programação procedural quanto programação orientada a objetos (os dois principais paradigmas em programação), tornando fácil a interação entre equipes de programadores acostumadas com quaisquer dessas filosofias.

Linguagem interpretada

Python não é compilado para rodar diretamente no computador; Python roda em um ambiente previamente instalado na máquina, e o programa é interpretado por esse ambiente no momento da execução.

Linguagem altamente portátil

Como Python não é compilado, não há diferenças no código de um programa feito para um computador e que será depois executado em outro tipo de máquina. O ambiente de programação Python, uma vez instalado em ambas as máquinas, vai proporcionar uma padronização de funções e recursos.

Linguagem script

A linguagem script é usualmente utilizada pelo sistema operacional para controlar outros programas e aplicativos. Python tem os recursos de uma linguagem script, o que significa que as aplicações feitas nessa linguagem podem diretamente executar e fazer uso dos resultados de outras aplicações instaladas no computador. Trata-se de uma facilidade para programadores que necessitam de uma integração entre aplicativos diferentes.

Baixe o ambiente de programação e arquivos de instalação, configuração, documentação, programas-exemplo, enfim, tudo o que quiser saber e fazer sobre Python através do link:

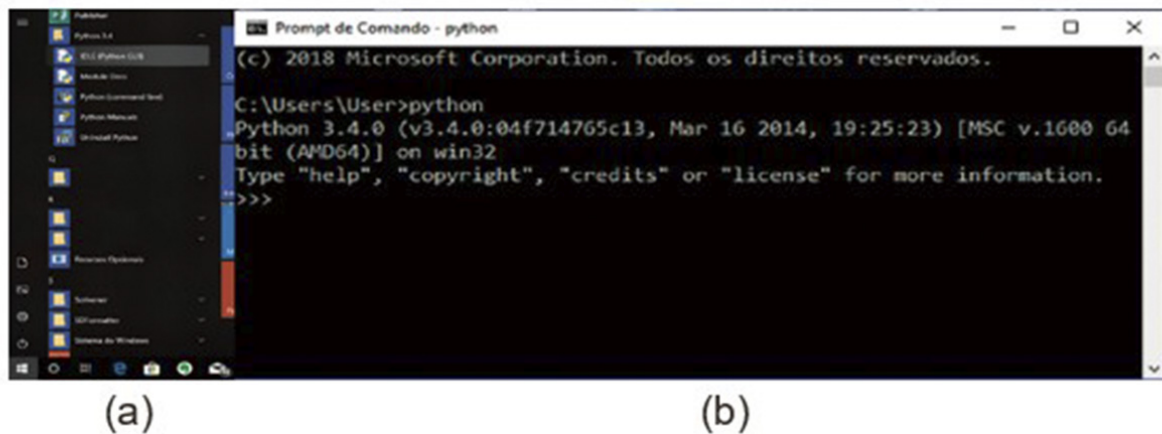
ACESSAR

Fundamentos da Linguagem Python

A linguagem Python vem com um ambiente próprio para o desenvolvimento de aplicações e que funciona no modo interativo e no modo programado, denominado *Integrated Development and Learning Environment* (IDLE).

Para iniciar o ambiente, escreva “python” na linha de comando da janela de comandos do seu computador ou clique no ícone IDLE (Python GUI) no menu da sua tela inicial. A figura a seguir mostra como iniciar a chamada do ambiente:

Figura 1 - Chamando o ambiente Python (a) no menu iniciar e (b) via console



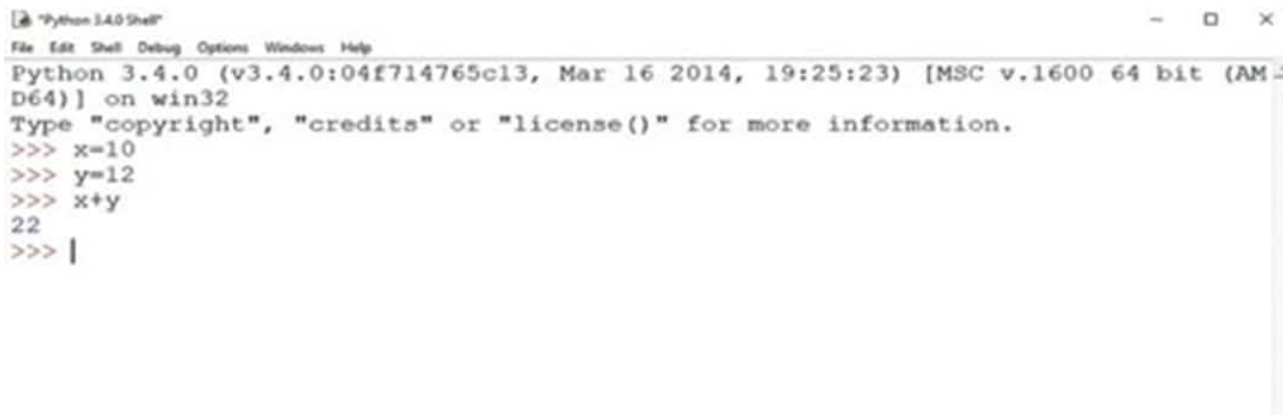
Fonte: o autor.

Modo Interativo

É a maneira mais fácil de trabalhar e se acostumar com a linguagem, conforme Maruch, S. e Maruch, A. (2006). No modo interativo, os comandos são executados imediatamente, exibindo na tela o resultado.

Veja o exemplo na figura abaixo:

Figura 2 - Exemplo do modo interativo

A screenshot of a Windows application window titled "Python 3.4.0 Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following content: "Python 3.4.0 (v3.4.0:04f714765c13, Mar 16 2014, 19:25:23) [MSC v.1600 64 bit (AMD64)] on win32", "Type \"copyright\", \"credits\" or \"license()\" for more information.", and an interactive session with the prompts ">>>" followed by the code "x=10", "y=12", "x+y", and the output "22". The cursor is on a new line after the last prompt. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

```
Python 3.4.0 (v3.4.0:04f714765c13, Mar 16 2014, 19:25:23) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x=10
>>> y=12
>>> x+y
22
>>> |
```

Fonte: o autor.

No exemplo, foi definida uma variável de nome `x` com valor 10, outra variável `y` com valor 12 e, depois, a soma `x + y`. O ambiente retornou o resultado imediatamente: 22.

O ambiente interativo pode ser utilizado para cálculos rápidos, testes de mesa ou mesmo para testar um bloco de comandos.

Variáveis e Expressões

Antes de dar continuidade ao modo programado do ambiente Python, é importante compreender um pouco mais sobre a linguagem Python.

Variáveis

As variáveis em Python são declaradas sempre iniciando com uma letra. Não podem ter espaços e são variáveis ditas case sensitive, isto é, variáveis com nomes iguais, mas, escritas em maiúsculas ou em minúsculas, são reconhecidas como diferentes.

Na sequência, confira exemplos de declarações de variáveis válidas:

```
x=10
nome="Maria"
Area_do_quadrado= Base * Altura
CATETO2=200.345
```

Confira agora exemplos de declarações de variáveis inválidas:

```
20Casal=300  
Nome de Família = "Torres"
```

Note também que, em Python, você não necessita definir o tipo de variável na declaração, como acontece com algumas linguagens. Basta fazer a atribuição do valor e a linguagem define diretamente o tipo, conforme afirma Lutz (2011).

No exemplo anterior, ao declarar `nome="Maria"`, Python já interpretou que `nome` será uma variável que conterá caracteres (os símbolos “ ” são a forma usual, na maioria das linguagens de programação, de sinalizar que `Maria` é uma palavra, um conjunto de letras, e não outra variável chamada `Maria`).

Instruções e Funções

Python possui instruções simples de escrever, tornando o código bem legível e de fácil entendimento. Além de instruções, Python possui um conjunto básico e extenso de funções à disposição do programador, bem como permite a instalação de bibliotecas com funções específicas.

Veja a seguir algumas instruções muito utilizadas na linguagem Python:

Entrada de dados - a entrada de dados, realizada pelo usuário via teclado, utiliza a instrução `input`:

```
Nome_da_Variavel = input('Mensagem ao usuário ')
```

No comando `input`, o usuário receberá a `Mensagem ao usuário`, orientando para fazer uma entrada de dados que será atribuída à variável `Nome_da _Variavel`.

Por exemplo: `Nome_do_ Aluno = input('Entre com o nome do aluno:')`

Mensagens e saída de dados - as mensagens dirigidas ao usuário, bem como aqueles resultados que serão apresentados durante e ao final do processamento, utilizam o comando `print`:

```
print('Mensagem ao usuário')
```

Acima, o comando `print` contém somente uma mensagem ao usuário, que será impressa na tela ou em papel.

```
print('Mensagem sobre alguma saída de dados',  
Nome_da_Variavel)
```

Nesse formato, o comando `print` emite uma mensagem ao usuário e, a seguir, o valor que está contido na variável. Esse é um formato usual para apresentação de resultados durante ou após o processamento.

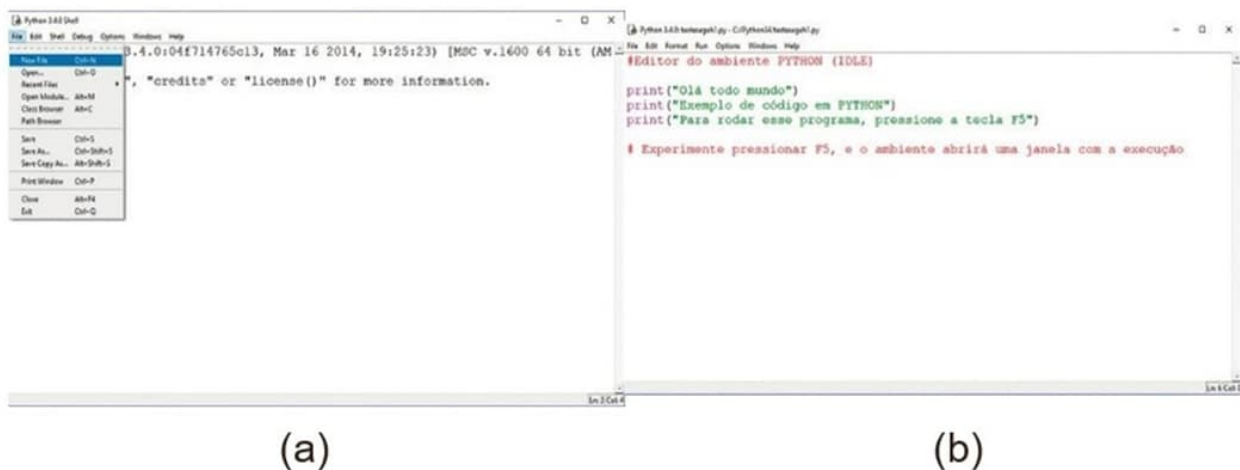
A linguagem Python é muito completa, atendendo várias necessidades de programação e diferentes tipos de aplicações, exigindo o aprendizado gradual de vários outros comandos e recursos.

Modo Programado

O modo programado do Python possui um editor de texto, permitindo a escrita do código diretamente.

Para utilizar esse modo, deve-se criar um arquivo novo com o editor do ambiente, diretamente no menu de cabeçalho do aplicativo IDLE (figura 3), conforme reitera Lutz (2011).

Figura 3 - Criação do programa: (a) chamando o editor e (b) editor ativo



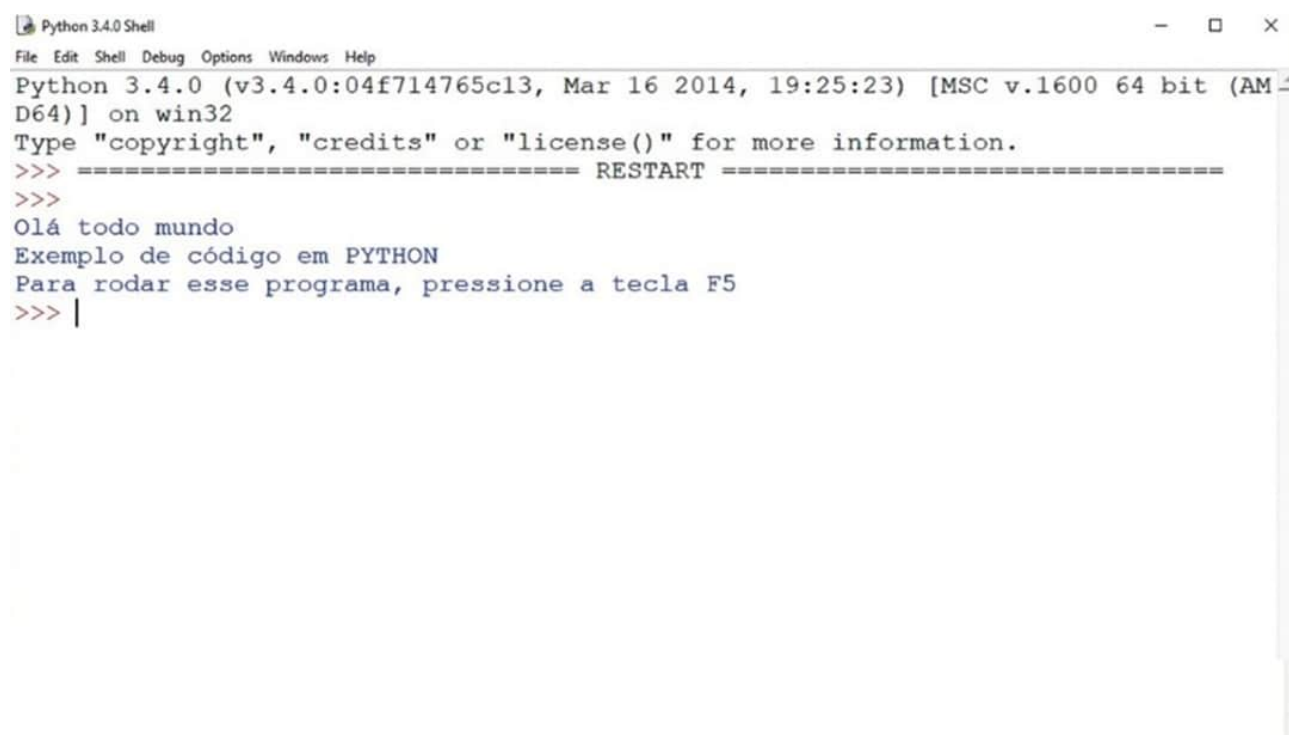
Fonte: o autor.

Na figura 3b, observe que o símbolo `#` marca o início de um comentário. Os comentários funcionam como mensagens que o programador utiliza em várias partes do código para descrever o funcionamento do programa. Essas mensagens não são processadas pela máquina.

A figura 3b também ilustra um primeiro programa em Python, que vai imprimir as mensagens contidas nas funções `print`.

Já a figura seguinte exibe o resultado após o uso da tecla F5:

Figura 4 - Resultado pós uso da tecla F5



```
Python 3.4.0 Shell
File Edit Shell Debug Options Windows Help
Python 3.4.0 (v3.4.0:04f714765c13, Mar 16 2014, 19:25:23) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Olá todo mundo
Exemplo de código em PYTHON
Para rodar esse programa, pressione a tecla F5
>>> |
```

Fonte: o autor.

Criando um programa em Python



AUTORIA
Cleber Jose Semensate Santos

Vamos utilizar os conceitos vistos até aqui para criarmos um programa em Python.

Enunciado

O plano diretor de desenvolvimento urbano de uma cidade determina qual é o percentual máximo de área destinado para garagem em relação à área total do terreno da casa, dependendo da localização desse terreno na cidade. Uma empresa de arquitetura está com vários contratos e necessita calcular rapidamente esse percentual, antes de iniciar os projetos. Faça um programa que recebe as medidas do terreno e da garagem e calcula o percentual de ocupação da área da garagem em relação ao terreno.

Algoritmo

O algoritmo para esse programa consiste no cálculo da área da garagem, no cálculo da área do terreno e em realizar o quociente entre os dois valores e multiplicar o resultado por 100, para uma resposta em percentual. Lembrando que: área (em metros quadrados) = largura (em metros) × profundidade (em metros). Observe, ainda, que a operação de multiplicação utiliza o símbolo *.

Veamos a sequência lógica desse algoritmo:

1. Entrar com o valor da largura da garagem em metros;
2. Entrar com o valor da profundidade da garagem em metros;
3. Calcular área da garagem = largura da garagem * profundidade da garagem;
4. Entrar com o valor da largura do terreno em metros;
5. Entrar com o valor da profundidade do terreno em metros;
6. Calcular área do terreno = largura do terreno * profundidade do terreno;
7. Calcular percentual de ocupação = (área da garagem / área do terreno) * 100;
8. Apresentar em tela o resultado.

Programa Python

O programa em Python é quase uma tradução direta do algoritmo (considerando a língua inglesa).

Observe a clareza do código:

```
Largura_Garagem=float(input("Entre com a largura da garagem
em
metros:"))

Profundidade_Garagem=float(input("Entre com a profundidade da
garagem em metros: "))

#Abaixo, cálculo da área da garagem
Area_Garagem=Largura_Garagem
* Profundidade_Garagem

Largura_Terreno=float(input("Entre com a largura do terreno
em
metros: "))

Profundidade_Terreno=float(input("Entre com a profundidade do
terreno em metros: "))

# Abaixo, cálculo da área do terreno
Area_Terreno=Largura_Terreno * Profundidade_Terreno

# Agora, cálculo do percentual de ocupação Percentual =
((Area_Garagem)/(Area_ Terreno))* 100 print("Percentual de
ocupação:",Percentual)
```

A figura a seguir apresenta o código no editor do ambiente Python e a execução do programa, com um exemplo em que a área da garagem é de 30 metros quadrados e a área do terreno da garagem é de 400 metros quadrados.

```

Python 3.4.0 Shell
File Edit Shell Debug Options Windows Help
Python 3.4.0 (v3.4.0:04f714765c13, Mar 16 2014, 19:25:23) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits()" or "license()" for more information.
>>> ===== RESTART =====
>>>
Olá todo mundo
Exemplo de código em PYTHON
Para rodar esse programa, pressione a tecla F5
>>> ===== RESTART =====
>>>
Entre com Largura da garagem em metros 5
Entre com a Profundidade da garagem em metros 6
Entre com Largura do Terreno em metros 20
Entre com a Profundidade do Terreno em metros 20
Percentual de ocupação: 7.5
>>> |

```

```

Python 3.4.0 Shell
File Edit Shell Debug Options Windows Help
Python 3.4.0 (v3.4.0:04f714765c13, Mar 16 2014, 19:25:23) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits()" or "license()" for more information.
>>> ===== RESTART =====
>>>
Olá todo mundo
Exemplo de código em PYTHON
Para rodar esse programa, pressione a tecla F5
>>> ===== RESTART =====
>>>
Entre com Largura da garagem em metros 5
Entre com a Profundidade da garagem em metros 6
Entre com Largura do Terreno em metros 20
Entre com a Profundidade do Terreno em metros 20
Percentual de ocupação: 7.5
>>> |

```

(a)
(b)

Pratique a criação de algoritmos, pois Python será o principal aliado para programar e gerar rapidamente os seus próprios aplicativos.



SAIBA MAIS

Programação e algoritmos: a base da computação

[...]

Ao contrário do que se apregoava há alguns anos, a atividade de programação deixou de ser uma arte para se tornar uma ciência, envolvendo um conjunto de princípios, técnicas e formalismos que visam a produção de produtos de *software* bem estruturados e confiáveis. Cite-se, dentre estes, os princípios da abstração e do encapsulamento e as técnicas de modularização e de programação estruturada.

Portanto, o estudo de programação não se restringe ao estudo de linguagens de programação. As linguagens de programação constituem-se em uma ferramenta de concretização de produtos de *software*, que representa o resultado da aplicação de uma série de conhecimentos que transformam a especificação da solução de um problema em um programa de computador que efetivamente resolve aquele problema. Deve ser dada ênfase aos aspectos funcionais e estruturais das linguagens, em detrimento aos detalhes de sintaxe. O estudo de linguagens deve ser precedido do estudo dos principais paradigmas de programação, notadamente a programação imperativa, a funcional, a baseada em lógica e a orientada a objetos.

O desenvolvimento de algoritmos e o estudo de estruturas de dados devem receber especial atenção na abordagem do tema programação. Igualmente, deve ser dada ênfase ao estudo das técnicas de especificação, projeto e validação de programas. Devese entender que ensinar programação não é apenas ensinar uma linguagem de programação. Este ensino envolve entender problemas e descrever formas de resolução, de maneira imparcial, para que então sejam codificadas em uma linguagem. Ou seja, somente após o aprendizado dos conceitos de algoritmo e fundamentos de lógica, o estudante pode travar contato com uma linguagem de programação. Além disso, o estudo de programação deve passar efetivamente pelo estudo de estruturas de dados.

[...]

Fonte: Santos e Costa (2002).



REFLITA

1) Coerência com os objetivos fundamentais: por coerência, entende-se que o professor deve: i) expressar claramente as ideias, os conceitos e as técnicas perante os alunos (se o professor coloca algoritmos confusos na lousa ou em transparências, ele não pode esperar algoritmos claros nas respostas dos alunos); ii) destacar a importância dos resultados teóricos e mostrar rigor formal toda vez que isto se fizer necessário; iii) valorizar o uso de técnicas na resolução de problemas.

2) Ênfase no pensamento crítico: deve-se ter um cuidado especial, pois os alunos que têm pouca maturidade matemática tendem a acreditar em qualquer demonstração. Tal comportamento deve ser desestimulado. É essencial que os alunos duvidem daquilo que é apresentado a eles e é com dúvidas saudáveis e sua resolução que a percepção da importância do resultado teórico poderá ser consolidada. Nesse sentido, considera-se um recurso valioso o conjunto de exercícios que pedem para os alunos identificarem falhas de argumentação, erros em algoritmos ou erros em notícias da imprensa;

[...]

Fonte: Santos e Costa (2002).



Conclusão - Unidade 1

Prezado(a) aluno(a), este capítulo foi uma introdução a algoritmos e programação, bem como foram abordados os princípios que embasam esse conteúdo.

Vimos que, para acompanhar, não foi necessário ter um conhecimento prévio de programação ou ser um *hacker* de computadores. Partiu-se do preceito de que o leitor sabe somente como interagir com o computador, ligar, desligar, instalar e utilizar programas e acessar a internet e pudemos perceber como tudo ocorreu bem.

Sendo assim, o intuito com este capítulo era o de estimular a sua curiosidade por esta disciplina, introduzindo os conceitos básicos, de forma que você pudesse entender a importância da programação e da linguagem de programação Python.

Além disso, vimos como preparar o ambiente, instalando o programa que utilizaremos – o PyCharm.

Nos vemos no próximo capítulo!



Livro

Introdução à Programação com Python: Algoritmos e Lógica de Programação Para Iniciantes

Autor: Nilo Ney Coutinho Menezes

Editora: Novatec

Sinopse: este livro é orientado ao iniciante em programação. Os conceitos básicos de programação, como expressões, variáveis, repetições, decisões, listas, dicionários, conjuntos, funções, arquivos, classes, objetos e banco de dados com SQLite 3 são apresentados um a um, com exemplos e exercícios. A obra visa a explorar a programação de computadores como ferramenta do dia a dia.

Filme

Introdução à Programação com Python - Microgenios - Dica de leitura #008

Ano: 2017

Sinopse: Vídeo com dica de leitura do canal “Microgenios”.

Referências

CORMEN, T. H. **Desmistificando algoritmos**. Rio de Janeiro: Elsevier; Campus, 2014.

CORMEN, T. H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier; Campus, 2012.

JET BRAINS SRO. **Getting started**. 2020. Disponível em: <https://www.jetbrains.com/pycharm/documentation/>. Acesso em: 4 mar. 2020.

LAW, S. **Filosofia: guia ilustrado** Zahar. 2. ed. Rio de Janeiro: Zahar, 2008.

LUTZ, M. **Programming Python**. 4. ed. Sebastopol: O'Reilly, 2011.

MARUCH, S.; MARUCH, A. **Python for dummies**. New York: John Wiley & Sons, 2006.

PYTHON SOFTWARE FOUNDATION. **Documentation**. s.d. Disponível em: <http://www.python.org>. Acesso em: 4 mar. 2020.

RUSSELL, S.; NORVIG, P. **Inteligência artificial**. 3. ed. Rio de Janeiro: Elsevier, 2013.

SANTOS, R. P.; COSTA, H. A. X. **TBC-AED e TBC-AED/Web**: Um desafio no ensino de algoritmos, estruturas de dados e programação. IV Workshop em educação em computação e informática do estado de Minas Gerais. 2005. Disponível em: <http://cos.ufrj.br/~rps/pub/completos/2005/WEIMIG.pdf>. Acesso em: 9 mar. 2020.



| Unidade 2

Tipo de dados e variáveis



AUTORIA
Cleber Jose Semensate Santos

Introdução

Olá, caro(a) aluno(a)!

Para poder operar, é necessário saber mais sobre a estrutura interna de um computador e como ele consegue processar as informações. Você já parou para pensar sobre os dados apresentados por um computador? Já imaginou como eles devem ser representados e como são interpretados e compilados por uma linguagem de programação?

Neste capítulo, você vai estudar sobre a representação interna de dados em um computador. Você vai identificar o que são constantes e variáveis e analisar como elas são utilizadas em programas de computador.

Veremos também que os operadores são símbolos utilizados para escrever expressões. Eles são essenciais para o desenvolvimento de algoritmos em qualquer linguagem de programação. Os operadores de atribuição possuem o papel de atribuir um determinado valor a uma variável. Já os operadores aritméticos consistem em operações matemáticas básicas, que podem ser aplicadas a variáveis numéricas.

Vamos lá! Será uma experiência única!

Representação interna de dados em um computador



AUTORIA
Cleber Jose Semensate Santos

Conforme já afirmava Papert (2008), o computador é uma máquina “burra”, utilizada por uma criança inteligente. Ao fazer essa afirmação, o autor se referia às capacidades da máquina, tendo em vista que a máquina depende do ser humano — mais especificamente, de um programador —, para que os programas criados possam executar as tarefas esperadas. Mas, afinal, o que é um computador e quais são seus componentes?

O computador é uma máquina que foi criada para receber e processar entradas e exibir saídas. Conforme Lee (2011), um computador é formado por uma unidade de processamento central (CPU, do inglês *central processing unit*), pela memória e por dispositivos de entrada e saída (E/S), além de uma tela (dispositivo de saída), um *mouse* (dispositivo de entrada) e um disco rígido (dispositivo de E/S). Essas partes do computador serão especificadas a seguir.

O cérebro do computador é a **CPU**, que é responsável por guardar e recuperar valores da memória e executar a aritmética básica e a lógica. A CPU tem o controle de qualquer instrução a ser executada, uma após a outra, podendo também migrar de um lugar para outro conforme as instruções que lhe são apresentadas.

Embora um computador seja uma máquina relevante, ele é limitado às ações do ser humano. Tudo o que é utilizado em um computador se baseia no trabalho desenvolvido por pessoas.

O computador também é composto de **memória**. É nesse local que os valores podem ser armazenados e recuperados. Trata-se de um dispositivo de armazenamento relativamente rápido, mas que perde seu conteúdo assim que o computador é desligado. A memória de um computador é dividida em diferentes locais; cada local dentro da memória tem um endereço e pode conter um valor.



O computador também é composto de **memória**. É nesse local que os valores podem ser armazenados e recuperados. Trata-se de um dispositivo de armazenamento relativamente rápido, mas que perde seu conteúdo assim que o computador é desligado. A memória de um computador é dividida em diferentes locais; cada local dentro da memória tem um endereço e pode conter um valor.

O **disco rígido** do computador também serve para armazenamento e possibilita que valores sejam armazenados e recuperados, porém se diferencia da memória e da CPU por ser mais lento. O seu diferencial é que consegue reter conteúdo mesmo quando a energia do computador está desligada.

O *mouse* e a tela do computador são considerados **dispositivos de entrada e saída**, respectivamente. O *mouse* realiza as ações recebidas (movimentos de seta do *mouse*, por exemplo) e envia para o computador; este recebe os dados, processa e produz alguma ação. Já a **tela** é responsável por exibir dados e informações que são processadas pelo computador, produzindo ações de saída. É responsável também por permitir que o usuário se comunique a partir das informações que são repassadas pelo monitor.

A estrutura interna de um computador opera a partir de um sistema numérico. Assim como utilizamos os números decimais 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9 para operar, representar o sistema decimal e as infinitas quantidades e valores que desejamos, os computadores digitais operam utilizando o **sistema binário**. Esse sistema possui apenas dois algarismos ou dígitos para fazer a representação de uma quantidade desejada: 0 e 1.

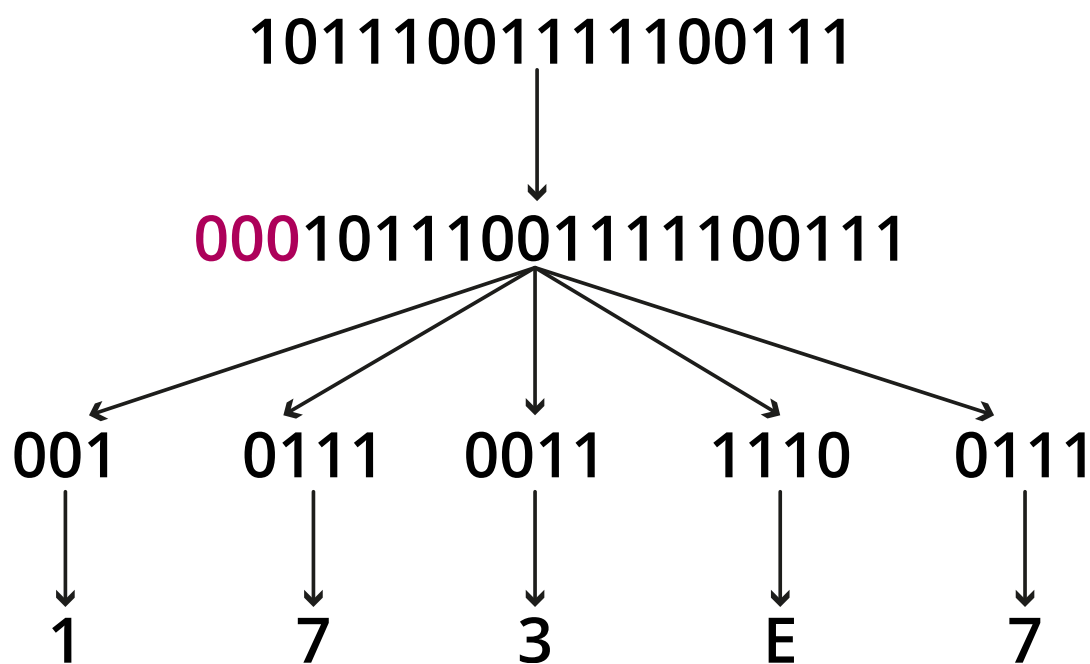
O sistema numérico binário, ou sistema na base 2, utiliza a noção de ligado/desligado, verdadeiro/falso ou 0/1. Segundo Edelweiss e Livi (2014, p. 21):



Internamente, os computadores digitais operam usando o sistema numérico binário, que utiliza apenas os símbolos 0 e 1. Na memória e nos dispositivos de armazenamento, o componente conceitual básico e a menor unidade de armazenamento de informação é o *bit*. *Bit* vem do inglês *binary digit*, ou seja, dígito binário, e um *bit* pode memorizar somente um entre dois valores: zero ou um. Qualquer valor numérico pode ser expresso por uma sucessão de bits usando o sistema de numeração binário.

No sistema binário, são necessários muitos algarismos para conseguir expressar uma informação, conforme pode ser visto na figura da sequência. Desse modo, outras formas de representação auxiliares também são utilizadas nos computadores. Para a representação dos caracteres, são utilizados códigos armazenados em conjuntos de *bits*.

Figura 1 - Sistema binário



Fonte: o autor.

De acordo com Edelweiss e Livi (2014), códigos mais simples armazenam os caracteres em *bytes* (conjuntos de 8 *bits*). Para códigos com representação de caracteres, existe uma convenção em que cada caractere tem associado a si uma sequência própria de zeros e uns.

Há três códigos que se destacam:

ASCII (7 bits por caractere)

American Standard Code for Information Interchange, utilizado por grande parte dos microcomputadores e em alguns periféricos de equipamentos de grande porte.

EBCDIC (8 bits por caractere)

Extended Binary Coded Decimal Interchange Code, utiliza um *byte* para a representação de cada caractere. Na representação do conjunto de caracteres ASCII padrão, o *bit* mais significativo (*bit* mais à esquerda) do *byte* é sempre igual a 0.

UNICODE (16, 32 ou mais bits)

Originado pela *Unicode Consortium*, visa a oportunizar que os computadores representem e manipulem textos de forma consistente nos mais variados sistemas de escrita existentes. Abrange mais de 100.000 caracteres, podendo utilizar um, dois ou mais *bytes* para representar caracteres.

Dentre os códigos mais utilizados estão o ASCII e o EBCDIC, que utilizam um *byte* para representar cada caractere. O conjunto de caracteres ASCII padrão, quando representado, tem o *bit* mais significativo (mais à esquerda) do *byte* sempre igual a zero.

Observe no quadro abaixo a representação dos caracteres A e Z nos dois códigos:

Quadro 1 - Representação dos caracteres A e Z nos códigos EBCDIC e ASCII

Caracteres	EBCDIC	ASCII
A	1100 0001	0100 0001
Z	1110 1001	0101 1010

Fonte: adaptado de Edelweiss e Livi (2014).

O próximo quadro apresenta as unidades de medidas que mensuram a memória principal e a capacidade de armazenamento do dispositivo. Os prefixos apresentados são os mesmos utilizados pelo sistema métrico de unidades de medida, porém, na informática, cada um deles possui um valor diferenciado.

Quadro 2 - Unidades de medida que indicam a capacidade de armazenamento dos dispositivos

K	quilo	mil	10^3
M	mega	milhão	10^6
G	giga	bilhão	10^9
T	tera	trilhão	10^{12}

Fonte: adaptado de Edelweiss e Livi (2014).

O sistema de numeração interno utilizado é binário (base 2). Sendo assim, as capacidades de armazenamento aparecem como potências de 2, conforme o exemplo apresentado no quadro da sequência:

Quadro 3 - Exemplos de capacidades de armazenamento representadas por potências de 2

K	1.024	2^{10}
M	1.048.576	2^{20}

Fonte: adaptado de Edelweiss e Livi (2014).

Definindo constantes, variáveis e seus tipos



AUTORIA
Cleber Jose Semensate Santos

Antes de analisar exemplos sobre constantes, variáveis e seus tipos, é importante compreender cada um desses elementos, conforme será apresentado a seguir.

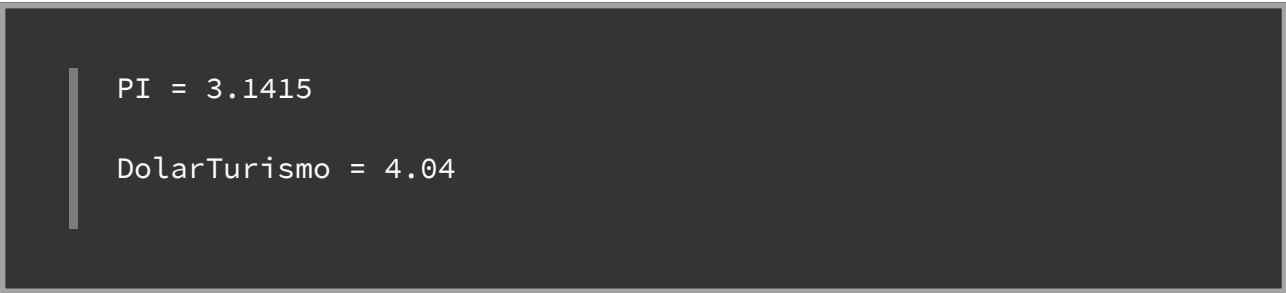
Constantes

As **constantes** representam dados ou objetos cujo valor permanece inalterado ao longo do tempo ou durante a execução do programa.

As constantes, no Python, geralmente são declaradas e disponibilizadas em um **módulo**. Esse módulo consiste em um arquivo novo, com variáveis, funções, etc., que são importadas para o arquivo principal. Dentro do módulo, lembre-se de que se deve escrever as constantes em letras maiúsculas e separar as palavras com o caractere sublinhado.

A figura a seguir traz exemplos de constantes:

Figura 2 - Exemplos de constantes

A imagem mostra um editor de código com um fundo escuro. Há duas linhas de código em branco: a primeira é 'PI = 3.1415' e a segunda é 'DolarTurismo = 4.04'.

```
PI = 3.1415  
  
DolarTurismo = 4.04
```

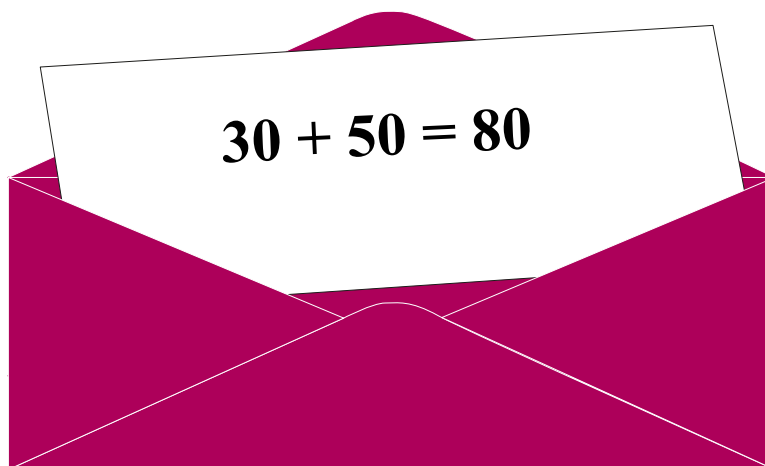
Fonte: o autor.

Variáveis e seus Tipos

Conforme Kalb (2016), uma **variável** é uma localização de memória nomeada que possui um determinado valor. Este pode variar conforme o tempo e, por isso, é denominado variável.

De modo geral, o Python armazena dados e nomes de variáveis de forma mais simples. Pode-se pensar na variável como se fosse um envelope ou uma caixa, em que pode ser adicionado um dado (um número, uma *string*, um *boolean*). Por exemplo, suponha que no envelope da figura 3 foi adicionado o número 30; mais tarde, nesse mesmo envelope, alguém adiciona o número 50, totalizando, assim, 80. O que você observou mudar? O envelope permanece o mesmo, porém, com o passar do tempo, os valores mudaram. Assim, uma variável fica sempre com o mesmo nome em um espaço de armazenamento, com um conteúdo (valor). O nome não muda, mas o conteúdo pode ser modificado no decorrer do tempo.

Figura 3 - Exemplo de variável



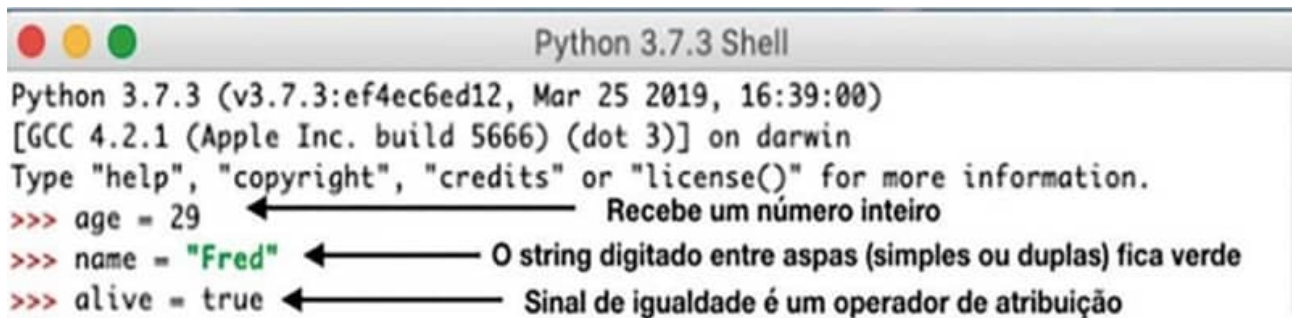
Fonte: o autor.

Agora que você já verificou os conceitos e as principais características de constantes e variáveis, serão apresentados alguns tópicos mais específicos sobre seu uso no Python.

No Python, as variáveis recebem um valor com uma **instrução de atribuição** (linha de código), que, de forma geral, pode ser apresentada da seguinte forma: `<variável> = <expressão>`. O que está dentro dos sinais de maior (<) e menor (>) deve ser substituído pelo nome da variável, sendo então `<variável>` um marcador de posição. O lado direito da igualdade pode ser substituído por um valor ou uma expressão.

Observe o exemplo da figura abaixo:

Figura 4 - Programação em Python



```
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:39:00)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>> age = 29
>>> name = "Fred"
>>> alive = true
```

Recebe um número inteiro

O string digitado entre aspas (simples ou duplas) fica verde

Sinal de igualdade é um operador de atribuição

Fonte: o autor.

Observe os *strings* digitados e o que foi atribuído:

```
>>> age = 29:
```

O sinal de igual na programação não corresponde à igualdade; ele é um **operador de atribuição** e mostra que o que está à direita do sinal de igual foi calculado. Esse resultado é atribuído à variável que está à esquerda do sinal. Ao digitar uma aspa de abertura, o interpretador (e você) percebe que será digitado um string; por isso, os caracteres ficam verdes. É importante ter o cuidado de abrir e fechar as aspas (simples ou duplas), para que o fechamento fique correspondido.

Cada variável deve possuir um nome. Para facilitar o trabalho, sugere-se que o nome da variável seja o mais descritivo possível, ou seja, que esteja relacionado com a função que ela vai exercer no programa. Dessa forma, podemos adotar algumas boas práticas para a **nomeação de variáveis** na linguagem Python, conforme descrito por Kalb (2016):

- Começar com uma letra ou sublinhado;
- Não começar com um dígito;
- Ter no máximo 256 caracteres;
- Letras, dígitos, sublinhados e cifrões podem ser inseridos;
- Não conter espaços e símbolos matemáticos (+, -, /, *, parênteses).

Além disso, em Python, existem duas convenções acerca da escrita para a nomenclatura de variáveis:

1. A primeira palavra deve ser toda em minúscula; para cada palavra adicional, somente a primeira letra deve ser maiúscula, e as demais, minúsculas. Exemplo: `essaVariavelNome`; `outraVariavel Nome`; `exemploVariavelNome`.
2. Escrever as palavras com um subtraço (`_`), também conhecido por underline, entre as palavras. Exemplo: `essa_variavel_nome`; `outra_variavel_nome`; `exemplo_variavel_nome`.

Assim, lembre-se de nomear as variáveis com nomes descritivos, que possam auxiliar e lembrar o que ela faz, observando também o uso de letras maiúsculas e minúsculas, de modo que o usuário consiga fazer a leitura. Além disso, procure identificar as variáveis com nomes curtos, pois estes geralmente são mais simples para a identificação, conforme sugere Kalb (2016).

Constantes e variáveis apropriadas para o uso em programas de computador



AUTORIA
Cleber Jose Semensate Santos

Ao iniciar uma programação, independentemente da linguagem, é necessário estar atento às constantes e variáveis que serão utilizadas, de modo que o compilador do programa consiga compilar os dados escritos.

De acordo com Kalb (2016), os compiladores de todas as linguagens alteram o código escrito na linguagem para instruções de nível inferior (com base em uns e zeros), para que o computador realmente possa entender o código. A compilação ocorre antes da execução do programa.

Cabe ressaltar que, para a nomeação de uma variável, é necessário observar as regras apresentadas anteriormente, pois elas podem auxiliar no momento da nomeação. Porém, existem outros detalhes importantes.

A forma geral de uma declaração de atribuição é:

```
<variavel> = <expressao>
```

O lado direito também pode conter variáveis, podendo não ser tão simples como os exemplos apresentados até então. Por exemplo:

```
>>> myAge = 31
>>> yourAge = myAge
>>>
>>> a = 1
```

Onde:

- >>> – caracteres do próprio prompt de comando;
- a – variável criada;
- = – não tem significado matemático; significa atribuição e não igualdade;
- 1 – valor atribuído para a variável.

Conforme instrui Melo (2019), essa operação pode ser avaliada de forma ainda mais detalhada, a partir da divisão em três passos.

1º passo

Gera-se na memória o objeto (dado) resultante do lado direito da atribuição (o número 1);

2º passo

Se a variável indicada do lado esquerdo da atribuição não existir no escopo (contexto) atual, cria-se essa variável (a variável a);

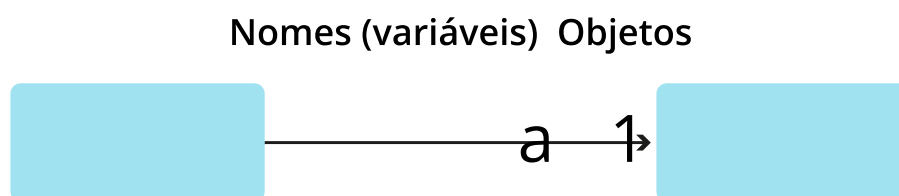
3º passo

A variável indicada no lado esquerdo da atribuição (a variável *a*) passa a apontar para o objeto gerado no 1º passo (o número 1).

A variável aponta para o número 1, conforme mostra a figura 5. Sendo assim, pode-se dizer que a variável está com valor 1 ou que está armazenando o valor 1. A atribuição sempre acontece da esquerda para a direita, portanto, a variável que recebe o resultado sempre deve estar na esquerda.

Segundo Melo (2019, p. 11), “sempre que for preciso utilizar a variável *a* para alguma operação, será resgatado o objeto para o qual ela aponta no momento da realização da operação”.

Figura 5 - Estado das variáveis e dos objetos na memória



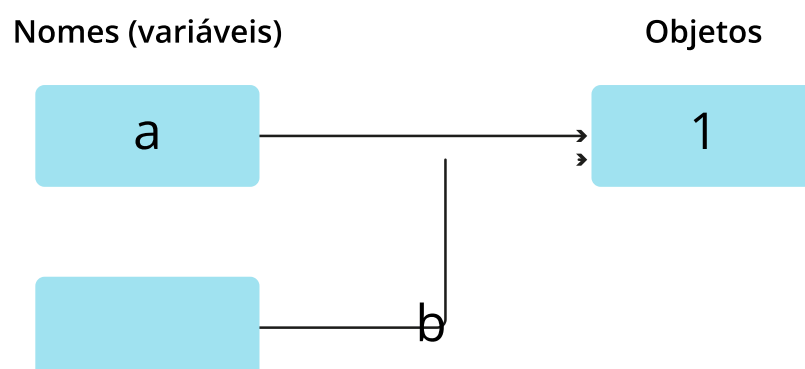
Fonte: o autor.

Agora, verifique outro exemplo, com base na seguinte expressão:

```
>>> b = a
```

Interpretando o comando, pode-se verificar que o objeto resultando à direita deve ser atribuído à variável que está à esquerda. Nesse caso, o resultado de *b* passa a indicar o mesmo que a variável *a* indica, conforme ilustrado seguinte na figura:

Figura 6 - Estado das variáveis e dos objetos na memória após atribuir *b = a*



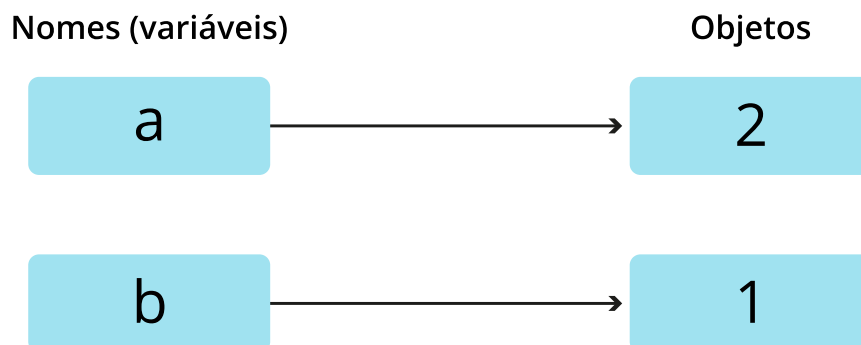
Fonte: o autor.

Na figura acima, *b* aponta para o mesmo objeto que *a* e a operação não gera relação entre *b* e *a*, sendo que estas são variáveis independentes.

Observe agora o caso de $a = 2$, apresentado na figura 7.

```
>>> a = 2
```

Figura 7 - Estado das variáveis e dos objetos na memória após atribuir $a = 2$



Fonte: o autor.

Nessa situação, a variável a passa a apontar para o objeto 2. Apenas o objeto em questão é apontado, ou seja, o interpretador não lembra mais que apontou para o objeto 1 anteriormente, pois, agora, o objeto em questão tem valor 2. No entanto, o valor da nova atribuição fez com que o valor atribuído anteriormente permanecesse o mesmo, ou seja, 1.

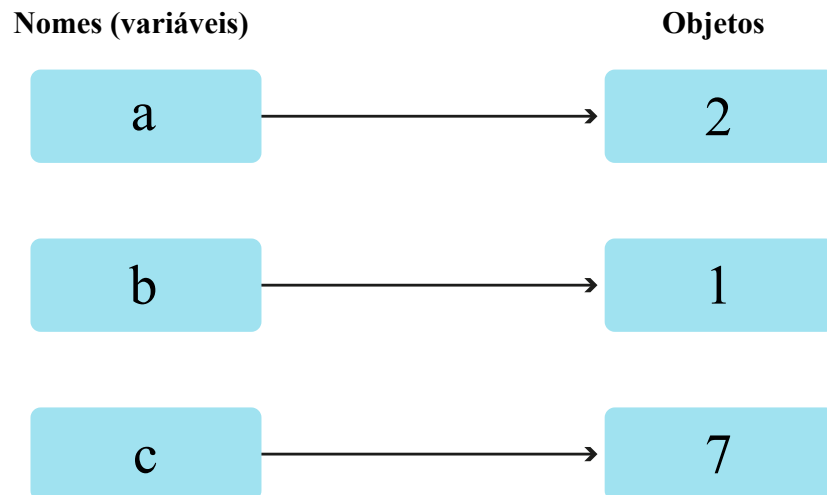
No Python, esses objetos não podem ser modificados após serem criados, o que garante que, caso haja a modificação de uma variável, as demais não serão afetadas. As variáveis podem apontar para um novo objeto a qualquer momento, porém não podem ser modificadas.

Outra atribuição possível de ser utilizada no lado direito são as expressões aritméticas. Observe o comando a seguir:

```
>>> c = b + 6
```

O comando acima possui uma variável c , que é apontada para o resultado da expressão $b + 6$, que é 7, pois o valor de b é 1 (figura 8).

Figura 8 - Estado das variáveis e dos objetos na memória após atribuir $c = b + 6$

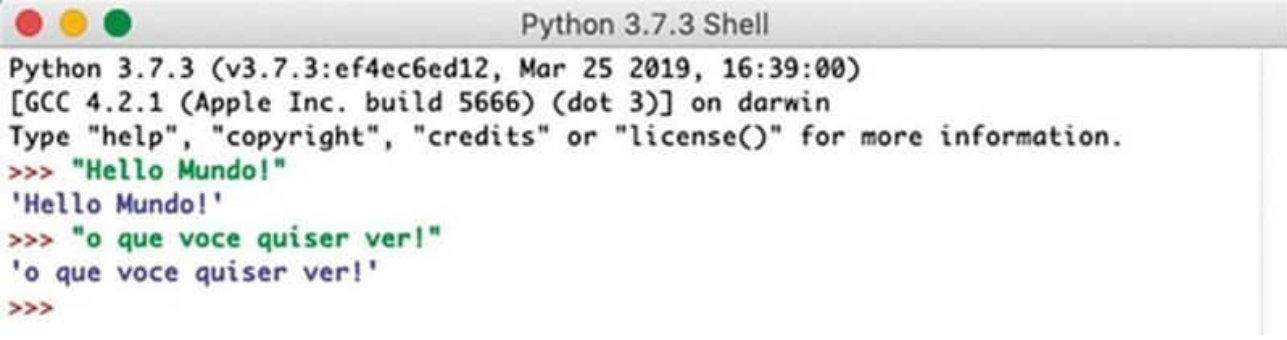


Fonte: o autor.

Conforme pôde ser observado na figura acima, a variável *c* está direcionada para o valor 7 (resultado da operação), mas sem saber que esse valor foi obtido a partir do valor atribuído a *b*, pois *b* e *c* são variáveis independentes. Qualquer mudança no objeto apontado por *b* não vai interferir em *c*.

Agora que você já sabe um pouco mais sobre as variáveis, pode começar a utilizar a programação em Python. Você pode iniciar escrevendo o que quiser, como mostra o código da figura a seguir, em que as mensagens “Hello Mundo!” e “o que você quiser ver” foram escritas.

Figura 9 – Mensagens escritas em linguagem de programação Python

A screenshot of a Python 3.7.3 Shell window. The title bar is grey with three colored window control buttons (red, yellow, green) on the left and the text 'Python 3.7.3 Shell' on the right. The main area is white and contains the following text: 'Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:39:00)' in grey, '[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin' in grey, and 'Type "help", "copyright", "credits" or "license()" for more information.' in grey. Below this, there are three lines of code: '>>> "Hello Mundo!"' in green, followed by the output ''Hello Mundo!'' in black, and '>>> "o que voce quiser ver!"' in green, followed by the output ''o que voce quiser ver!'' in black. The prompt '>>>' is shown at the end of the last line.

```
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:39:00)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>> "Hello Mundo!"
'Hello Mundo!'
>>> "o que voce quiser ver!"
'o que voce quiser ver!'
>>>
```

Fonte: o autor.

Objetos que não podem ser alterados



AUTORIA
Cleber Jose Semensate Santos

Conforme já mencionado, há objetos que não podem ser modificados, apenas excluídos. Portanto, para modificar um valor imutável pertencente a uma variável, necessita-se gerar um novo objeto com o valor devido e utilizar uma operação de atribuição para que ele seja mostrado pela variável, conforme com Melo (2019).

Os **objetos imutáveis** (listas, conjuntos e dicionários) admitem sua modificação na memória, o que exige mais atenção ao manipulá-los. Os dados numéricos são imutáveis na linguagem de programação Python.



ATENÇÃO

A seguir, são listadas algumas classes de tipos básicos imutáveis:

bool: só assume dois valores, `True` (verdadeiro) e `False` (falso). O Python associa o número 1 ao valor `True` e 0 ao valor `False`.

int: esses objetos possuem precisão arbitrária, ou seja, podem exibir números com quantidade arbitrária de dígitos, sendo possível calcular números cujo resultado é um valor bem considerável.

float: representa números reais racionais, que permitem a representação na codificação ponto flutuante, com 64 dígitos binários de precisão. Nessa classe, usa-se o caractere “.” para separar a parte inteira da decimal, e não “,”.

complex: representa números complexos. A letra “j”, maiúscula ou minúscula, é utilizada para representar o número imaginário.

None (`NoneType`): objeto que satisfaz como lugar reservado vazio; é um objeto que apenas ocupa espaço. O `None` pode ser utilizado para representar algo que, no momento, é desconhecido, mas que futuramente será conhecido.

str: dado utilizado para representar texto; sequências de caracteres (*strings*) são declaradas a partir do uso de aspas simples ou duplas, de modo equivalente.

tuple: são apresentadas com parênteses (opcionais), e os elementos são separados por “,”.

type: objetos utilizados para a manipulação de tipos.

Comandos de atribuição e operadores aritméticos



AUTORIA
Cleber Jose Semensate Santos

Os **operadores** são símbolos utilizados para realizar alguma operação com as variáveis do programa. Lembre-se de que as **variáveis** consistem em um pequeno espaço na memória, em que podemos armazenar alguma informação, seja ela um número ou um texto, como afirma Santos (2018).

O **operador de atribuição** mais simples que existe na linguagem Python é o operador de **igualdade**, cujo símbolo é o **=**. Por meio dele, podemos armazenar uma nova informação dentro de uma variável, de acordo com a sintaxe básica representada na figura abaixo:

Figura 10 - Representação da sintaxe básica do operador de atribuição de igualdade em Python



Fonte: o autor.

Além do **operador de atribuição (=)**, existem ainda as quatro **operações aritméticas básicas**: soma, subtração, multiplicação e divisão, representadas pelos operadores **+**, **-**, ***** e **/**, respectivamente. Todos esses operadores devem ser utilizados de forma análoga à forma ilustrada na figura 10.

Na figura 11, é possível observar um exemplo de código com essas operações. Observe que cada operação foi realizada e armazenada em uma variável com o nome da operação realizada. Para exibir o valor da variável na tela, foi utilizado o comando `print`, que será estudado nas próximas lições.

Figura 11 - Exemplo de uso dos operadores aritméticos básicos da linguagem Python

```
>>> soma = 1 + 2
>>> print(soma)
3
>>> subtracao = 4 - 3
>>> print(subtracao)
1
>>> multiplicacao = 2 * 3
>>> print(multiplicacao)
6
>>> divisao = 6 / 2
>>> print(divisao)
3.0
```

Fonte: o autor.

Além das operações aritméticas básicas, existem, ainda, os **operadores de potenciação** (**), o operador resto da divisão, também conhecido como **módulo (%)**, e o **operador parte inteira da divisão (//)**.

No código ilustrado na figura 12, é possível observar um exemplo de operação desses operadores adicionais:

Figura 12 - Exemplo de uso dos operadores aritméticos adicionais da linguagem Python

```
>>> potencia = 2**3
>>> print(potencia)
8
>>> resto = 10 % 3
>>> print(resto)
1
>>> parteInteira = 10 // 3
>>> print(parteInteira)
3
```

Fonte: o autor.

Ordem de Interpretação dos Operadores

Na avaliação de uma expressão aritmética, é importante lembrar que alguns desses operadores serão avaliados antes dos outros. Na linguagem Python, a **ordem de prioridade** começa nos operadores de potência (*), de módulo (%) e de resto da parte inteira (//). Após esses operadores terem sido avaliados, serão avaliados os operadores de multiplicação e divisão, para, só então, serem avaliados os operadores de soma e subtração.

Da mesma forma que, em uma expressão aritmética, é possível empregar os parênteses, os colchetes e as chaves para separar a expressão matemática em blocos, é possível fazer isso na linguagem Python. Porém, nessa linguagem, é possível usar apenas os **parênteses**. Os colchetes e as chaves possuem funções especiais na linguagem Python. A **ordem de interpretação** da expressão será sempre do parêntese mais interno para o mais externo. Por exemplo, na expressão $1 * 2 - (3 + 4)$, será avaliada primeiro a soma de $3 + 4$, depois a multiplicação de $1 * 2$, para, só então, ser avaliada a subtração do resultado da multiplicação com o resultado da soma.

Operadores de atribuição compostos

A linguagem Python possui um conjunto de operadores que facilitam operações, como a soma de um número qualquer a uma variável. Por exemplo, se tivermos uma variável auxiliar, cujo valor consiste em um número inteiro qualquer, e queremos adicionar 3 a ela, podemos escrever `auxiliar = auxiliar + 3`.

Observe que, nesse exemplo, apesar de simples, foi necessário escrever o nome da variável duas vezes. Isso não só é mais cansativo, como pode ser uma fonte de erros. Além disso, deixa o código final mais difícil de ler. Nesse caso, podemos lançar mão dos **operadores de atribuição compostos**, conforme sugere Banin (2018).

O mesmo exemplo abordado acima poderia ser escrito da forma `auxiliar += 3`, sendo ambas as abordagens iguais, em termos funcionais. Porém, mediante o uso desse operador, o código fica mais simples, mais limpo e menos sujeito a erros de digitação.

Como você pode perceber, os operadores de atribuição compostos permitem uma escrita de código mais limpa, mais clara e muito mais eficiente. Além do operador de atribuição composto `+=`, a linguagem Python possui ainda os operadores `-=`, `*=`, `/=` e `%=`. Cada um deles possui um funcionamento análogo ao do operador `+=`.

Na figura 13, você pode observar como se dá o funcionamento de cada um desses operadores de atribuição compostos.

Figura 13 - Funcionamento dos operadores de atribuição compostos

```
>>> auxiliar = 3
>>> auxiliar = auxiliar + 2
>>> print(auxiliar)
5
>>> auxiliar = 3
>>> auxiliar += 2
>>> print(auxiliar)
5
```

Fonte: o autor.



SAIBA MAIS

Aprendizagem de algoritmos

Um dos problemas mais relatados pela literatura, e também facilmente vivenciado por qualquer professor que já tenha alguma experiência no ensino de algoritmos, é quando este ensino ocorre sob a falta de recursos computacionais onde o aluno possa receber *feedback* imediato das suas tentativas em resolver os problemas propostos. Portanto, o aluno necessita de ambientes que possa experimentar, simular, investigar possibilidades e, sobretudo, errar bastante para que tenha percepção sobre os limites de um algoritmo.

A falta de ambientes de teste logo no início da aprendizagem de algoritmo supõe que o estudante já possui um excelente nível de abstração em conjunto com um bom raciocínio hipotético-dedutivo, uma vez que ele deve conseguir enxergar os resultados de um programa apenas inferindo sobre o código. É claro que esta abstração deverá ser desenvolvida ao longo de sua trajetória, pois bons programadores são também bons investigadores de erros e esta formação se faz, por exemplo, com exercícios do tipo “teste de mesa”, nos quais o estudante percorre o programa fazendo o papel do computador.

A atividade de programar possui uma cognição muito rica, pois faz o aluno percorrer o ciclo descrever-executar-refletir-depurar apresentado há bastante tempo por Papert (1994) e muito conhecido e aplicado pela comunidade acadêmica. Este ciclo é muito útil na formação do profissional, porque o faz desenvolver competências associadas às necessidades atuais do mercado de trabalho como as capacidades de planejar, antecipar e simular resultados, entre outros.

[...]

Fonte: Cristovão (2008).



REFLITA

1) Existe a crença de que só se pode programar o que se compreende perfeitamente. Essa crença ignora a evidência de que a programação, como qualquer outra forma de escrita, é um processo experimental. Programamos, como redigimos, não porque compreendemos, mas para chegar a compreender. (Joseph Weizenbaum) citado por (Vitale, 1991).

2) O domínio da linguagem adotada para se escrever o algoritmo é fundamental para que o autor consiga se expressar de forma correta para solucionar um problema. Assim, o grupo é estimulado pelo professor a explorar, questionar e validar a solução apresentada pelo colega. Essa verbalização constrói, sob a intervenção dos componentes da turma, progressiva e interativamente, a formalização considerada satisfatória pelo grupo. O papel do professor nessa etapa é o de minimizar o nível de competitividade e manter o grupo em ação colaborativa e investigativa.

[...]

Fonte: Cristovão (2008).

Conclusão - Unidade 2

Olá, aluno(a)!

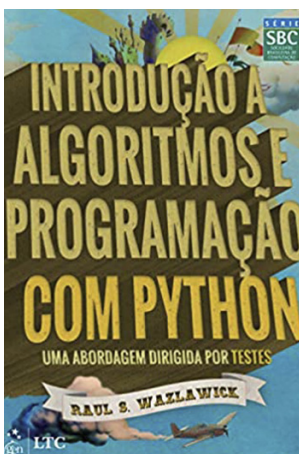
Neste capítulo vimos que, para poder operar, é necessário saber mais sobre a estrutura interna de um computador e como ele consegue processar as informações. Também vimos como os dados são apresentados por um computador e como eles devem ser representados, interpretados e compilados por uma linguagem de programação.

Além disso, vimos ainda como estudar sobre a representação interna de dados em um computador. Identificamos o que são constantes e variáveis e analisamos como elas são utilizadas em programas de computador.

Por fim, vimos também que os operadores são símbolos utilizados para escrever expressões; eles são essenciais para o desenvolvimento de algoritmos em qualquer linguagem de programação. Além de termos visto que os operadores de atribuição possuem o papel de atribuir um determinado valor a uma variável. Entendemos, ainda, que os operadores aritméticos consistem em operações matemáticas básicas, que podem ser aplicadas a variáveis numéricas.

Parabéns por mais esta etapa e continue firme nos estudos!

Material Complementar



Livro

Introdução aos Algoritmos e Programação com Python

Autor: Raul Wazlawick

Editora: Elsevier

Sinopse: o projeto ensina o pensamento algorítmico juntamente com a capacidade de programação de computadores usando a linguagem Python. Python vem se tornando a linguagem dominante no mundo todo quando o assunto é aprender a programar. Sua sintaxe é limpa, simples, concisa e, portanto, muito mais fácil de aprender do que outras linguagens. Cada vez mais universidades e escolas têm percebido que ela é a melhor opção quando se trata de ensino de algoritmos e programação. O conteúdo apresenta uma introdução ao pensamento algorítmico usando Python como linguagem e voltado para o iniciante, ou seja, para pessoas que não tenham nenhum conhecimento sobre algoritmos e programação.



Maratona de Programação em Python Algoritmos Básicos

Ano: 2017

Sinopse: Veja alguns algoritmos básicos na linguagem Python, para treinar e, ao longo dos estudos, adquirir habilidades em programação. São algoritmos simples, como as quatro operações aritméticas, entrada e saída de valores, conversão de tipos de dados de float (número real) ou integer (número inteiro) para string (texto), média aritmética entre valores para cálculo de média de aluno para verificar se está aprovado vai fazer exame final ou se o aluno está reprovado.

Referências

BANIN, S. L. **Python 3**: conceitos e aplicações: uma abordagem didática. São Paulo: Érica, 2018.

CRISTOVÃO, H. M. Aprendizagem de algoritmos num contexto significativo e motivador: um relato de experiência. SBC (2008): 30. **Anais do XXVIII Congresso da SBC**. WEI – Workshop sobre Educação em Computação Unidade de Computação e Sistemas Faculdades Integradas Espírito-santenses (FAESA). Julho 2008. Disponível em: <http://www2.sbc.org.br/csbc2008/pdf/arq0123.pdf>. Acesso em: 6 mar. 2020.

EDELWEISS, N.; LIVI, M. A. C. **Algoritmos e programação**: com exemplos em Pascal e C. Porto Alegre: Bookman, 2014.

KALB, I. **Learn to program with Python**. Mountain View: Apress, 2016.

LEE, K. D. **Python programming fundamentals**. New York; Dordrecht; Heidelberg; London: Springer, 2011.

MELO, W. **IUP – Introdução ao universo da programação com Python**: um livro aberto para aprender programação. Uberlândia: Faculdade de Computação, Universidade Federal de Uberlândia, 2019. Disponível em: http://www.facom.ufu.br/~wendelmelo/meu_material/introducao_programacao_python_wendel_melo.pdf. Acesso em: 18 fev. 2020.

MUELLER, J. P. **Começando a programar em Python para leigos**. 1ªed. Starlin, 2016.

PAPERT, S. **A máquina das crianças**: repensando a escola na era da informática. Porto Alegre: Artmed, 2008.

SANTOS, M. G. **Algoritmos e programação**. Porto Alegre, 2018.



| Unidade 3

Atribuição e operadores aritméticos



AUTORIA
Cleber Jose Semensate Santos

Introdução

Olá, caro(a) aluno(a)!

Como sabemos, o mundo está profundamente digitalizado. É inquestionável o fato de que os sistemas digitais estão constantemente tomando decisões, em frações de segundos. Uma importante parcela dessas decisões pode resultar em danos e acidentes terríveis, bem como na perda de vidas humanas.

A lógica de decisão adotada por uma máquina está registrada em seus circuitos de memória, ou seja, tal lógica foi previamente elaborada e programada na máquina. Em outras palavras, um ser humano com inteligência – mais especificamente, um programador – desenvolveu e implantou na máquina o processo de decisão e os procedimentos subsequentes, sendo ele, então, o responsável direto pelas ações da máquina, uma vez que máquinas não pensam, não raciocinam e não são inteligentes.

Assim, formalizar o processo de decisão que envolve qualquer tarefa que desejamos que a máquina realize constitui um importante e necessário conhecimento do programador. Este, então, utilizará operadores lógicos e instruções específicas para implantar suas decisões.

Neste capítulo, você vai estudar a forma de programar o processo de decisão na máquina, aprendendo sobre as estruturas lógicas que governam esse processo. Você também vai analisar em detalhes o comando `if`, empregado na programação de computadores, e suas formas de utilização. Por fim, você vai verificar como resolver problemas utilizando a instrução `if` na linguagem de programação Python.

Definição dos operadores lógicos e relacionais



AUTORIA
Cleber Jose Semensate Santos

A essência de qualquer decisão é, em última análise, uma **comparação**. A todo momento, realizamos comparações dos mais variados tipos, envolvendo os mais diferentes assuntos, tópicos, ações, desejos, dentre outras necessidades. Podemos classificar nossos modos de comparação nas seguintes categorias, conforme aponta Law (2008):

- Equivalência ou similaridade: igual ou diferente;
- Qualidade: melhor ou pior;
- Dimensão: maior ou menor;
- Quantidade: mais ou menos, muito ou pouco.

Em programação, tais formas de comparação estarão presentes na comparação envolvida no **procedimento de decisão** que será programado. No entanto, observe que algumas das categorias de comparação são claramente subjetivas, isto é, exigem uma interpretação consciente e inteligente para serem realizadas. Por exemplo, quando dizemos que “é melhor beber suco de laranja sem açúcar do que suco de limão sem açúcar”, estamos estabelecendo a opinião do autor da sentença sobre a experiência dele em beber esses sucos. No entanto, lidando com a realidade digital das máquinas, que não são inteligentes nem têm experiências de vida, não há contexto subjetivo ou experiência para a representação desse tipo de sentença.

Desse modo, um aspecto importante da programação é que qualquer comparação envolve a quantificação de algum dado que caracterize a comparação necessária para a decisão. Voltando ao exemplo, para programar de algum modo a decisão sobre o suco de laranja e o suco de limão, pode-se usar a medida de acidez, em função da quantidade percentual de ácido cítrico presente nos sucos. Então, uma forma de representar a capacidade de ingestão do suco sem açúcar na sentença “melhor beber suco de laranja do que suco de limão” é dizendo que “beber sem açúcar implica beber suco com baixa acidez e a acidez do suco de laranja é menor do que a acidez do suco de limão”, conforme teorizam Russell e Norvig (2013).



Operadores relacionais



AUTORIA
Cleber Jose Semensate Santos

Para representar as sentenças da declaração da comparação, usualmente denominada **teste lógico** ou **condição lógica**, utilizam-se os operadores relacionais:

- $A = B$, A igual a B;
- $A \neq B$, A diferente de B;
- $A > B$, A maior do que B;
- $A \geq B$, A maior ou igual a B;
- $A < B$, A menor do que B;
- $A \leq B$, A menor ou igual a B;

Em lógica, A e B podem ser quaisquer conceitos, abstratos ou não. Em programação, no entanto, A e B são **variáveis ou constantes** que serão avaliadas no momento em que a instrução de comparação for executada, conforme afirma Lutz (2011).

Em Python, bem como em outras conhecidas linguagens de programação, os operadores relacionais são representados no código conforme mostra o quadro abaixo:

Quadro 1 - Operadores relacionais

Operador relacional	Descrição	Representação em Python
$A = B$	A igual a B	<code>A == B</code>
$A \neq B$	A diferente de B	<code>A != B</code>
$A > B$	A maior do que B	<code>A > B</code>
$A \geq B$	A maior ou igual a B	<code>A >= B</code>
$A < B$	A menor do que B	<code>A < B</code>
$A \leq B$	A menor ou igual a B	<code>A <= B</code>

Fonte: o autor.

O operador relacional será utilizado para representar quaisquer condições de decisão, sendo a forma como será transcrita em código a condição lógica de decisão de qualquer enunciado.

Considere o seguinte exemplo:

Sentença: a idade mínima para entrada no cinema é 18 anos.

Representação: PERMISSÃO PARA INGRESSO - IDADE $\geq$ 18.

Note que tal condição lógica poderia fazer parte, por exemplo, da tomada de decisão em um sistema automatizado de acesso a uma sala de cinema. Assim, para esse exemplo, suponha um sistema que obtém a idade por meio da leitura da carteira de estudante ou de outra forma de identificação. Um fragmento do algoritmo referente à decisão seria:

LER IDADE DO ESTUDANTE SE A IDADE $\geq$ 18 ANOS:

ENTÃO

ACESSO PERMITIDO (ACIONAR ABERTURA DE PORTA DE ACESSO E MENSAGEM “ACESSO PERMITIDO”)

SENÃO

ACESSO NEGADO (FECHAR OU MANTER FECHADA A PORTA DE ACESSO E MENSAGEM “IDADE MÍNIMA NÃO ATENDIDA – ACESSO NEGADO”)

Note que a comparação está na condição lógica $IDADE \geq 18$ ANOS e o resultado da comparação leva à decisão de permitir ou não o acesso à sala de cinema.

Operadores lógicos



AUTORIA

Cleber Jose Semensate Santos

Para a construção de testes lógicos mais completos, envolvendo mais de uma condição comparativa, utilizaremos os operadores lógicos OU, E e NÃO (NEGADO ou NEGAÇÃO). Assim, tais operadores lógicos servirão como conectores entre condições comparativas, representando de forma completa uma dada sentença de decisão, conforme apontam Russell e Norvig (2013).

A seguir, serão abordados os operadores lógicos OU, E e NEGAÇÃO.

Operador OU (A ou B)

Esse operador é muito utilizado em lógica e constitui a operação que conecta duas condições, A e B, que podem ocorrer **conjuntamente ou de forma alternada**. Quando se diz A **OU** B, significa dizer que:

1. A condição A e a condição B podem ambas ser verdadeiras;
2. A pode ser verdadeira e B falsa;
3. A pode ser falsa e B verdadeira.

Vejamos um exemplo:

Sentença: o animal bovino possui cores branca ou preta.

Representação: BOVINOS: (Cor = BRANCA) OU (Cor = PRETA).

No exemplo, um BOVINO poderá ser de cor BRANCA, poderá ser de cor PRETA e poderá ser de duas cores, BRANCA e PRETA (bovinos malhados).

No quadro seguinte, podemos verificar a tabela verdade do operador, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como condição VERDADEIRA e F como condição FALSA.

Quadro 2 - Tabela verdade para o operador lógico OU

A	B	A ou B
F	F	F
F	V	V
V	F	V
V	V	V

Fonte: o autor.

O operador A ou B somente é falso (o resultado é F) quando **A é falso e B é falso**.

Operador E (A e B)

Trata-se do operador lógico que relaciona duas condições, A e B, que ocorrem **somente juntas**. Quando se diz A **E** B, significa dizer que A acontece (é verdadeiro) e B também necessariamente acontece (é verdadeiro).

Vejamos um exemplo:

Sentença: toda autorização de pagamento deve ser assinada pelo presidente e pelo tesoureiro.

Representação: AUTORIZAR PAGAMENTO - (ASSINATURA DO PRESIDENTE = SIM) E (ASSINATURA DO TESOUREIRO = SIM).

No exemplo, o presidente e o tesoureiro devem ambos assinar a autorização; isto é, as duas condições devem ser satisfeitas. Se somente um deles, isoladamente, assinar o documento, sem a assinatura do outro, não haverá autorização.

O quadro abaixo apresenta a tabela verdade do operador, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como fato VERDADEIRO e F como fato FALSO.

Quadro 3 - Tabela verdade para o operador lógico E

A	B	A e B
F	F	F
F	V	F
V	F	F
V	V	V

Fonte: o autor.

O operador $A \text{ e } B$ somente é verdadeiro (o resultado é V) quando **A é verdadeiro** e **B é verdadeiro**.

Operador de Negação (não A)

Este é o operador mais simples utilizado em lógica e consiste na operação lógica de inversão do estado lógico da condição. Quando se diz não A, ou a negação de A, significa dizer que, se A existe ou é verdadeiro, não A inexistente ou é falso.

Vejamos um exemplo:

Sentença: acionar a admissão de água na caixa quando ela estiver vazia.

Representação: admissão de água caixa - (não CHEIA).

No exemplo, a caixa d'água deverá receber água quando estiver vazia, isto é, a condição contrária ou negada de cheia.

No próximo quadro, pode-se conferir a tabela verdade do operador, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como fato VERDADEIRO e F como fato FALSO.

Quadro 4 – Tabela verdade para o operador lógico de negação (não A)

A	não A
F	V
V	F

Fonte: o autor.

O quadro a seguir resume os operadores lógicos e suas representações e o respectivo código de programa da linguagem Python:

Quadro 5 - Operadores lógicos e sua representação em Python

Operador lógico	Descrição	Representação em Python
A ou B	A OU B	A or B
A e B	A E B	A and B
não A	NÃO A	not A

Fonte: o autor.

Segundo Lutz (2011), utilizando os operadores relacionais e os operadores lógicos, pode-se representar testes lógicos complexos, envolvendo, em um único teste, situações com mais de uma condição relacional, conectadas por meio de operadores lógicos.

Vejamos um exemplo:

Sentença: em um dado país, os homens devem se apresentar ao serviço militar aos 18 anos, enquanto as mulheres podem se apresentar com idade maior ou igual a 18 anos e menor do que 36 anos.

Representação: SERVIÇO MILITAR – (HOMENS; IDADE = 18 ANOS) OU (MULHERES; IDADE $\geq$ 18 ANOS E IDADE < 36 ANOS).

Na próxima seção, vamos estudar e utilizar a instrução if para a programação de decisões em linguagem Python.

Analizando os comandos IF e IF...ELSE



AUTORIA
Cleber Jose Semensate Santos

O comando condicional `if`, do inglês “se”, é encontrado em praticamente todas as linguagens de programação de alto nível, sendo talvez um dos mais importantes e, sem dúvida, um dos mais antigos comandos existentes no conjunto de instruções dos primeiros computadores.

Estrutura Condicional `if` Simples

A instrução `if` é a instrução básica em programação para a construção de decisões que a máquina vai realizar durante a execução do programa.

Observe que o processo de decisão altera a sequência das instruções que serão realizadas pela máquina, também chamada de **fluxo de processamento**.

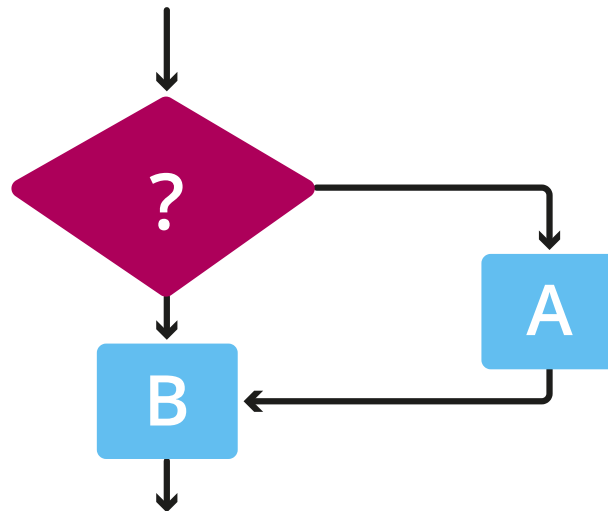
Veja o formato em Python da instrução `if` simples:

```
if condição lógica: #Bloco A
    Bloco de comandos executados caso a condição lógica seja
    verdadeira
#Bloco B
Comandos executados após o teste
```

Observe ainda que, em Python, a tabulação e o alinhamento das instruções definem os blocos de cada parte do comando `if`. Assim, se a condição lógica for VERDADEIRA, o Bloco A de comandos será executado e, após o término das instruções que compõem o Bloco A, o Bloco B será executado. Se, no entanto, a condição lógica for FALSA, somente o Bloco B de instruções será executado.

A figura da sequência exibe um diagrama com a representação gráfica dessa situação:

Figura 1 - Estrutura condicional if simples



Fonte: o autor.

Vejamos um exemplo:

```
if temperatura>25:
    print("Ambiente quente, ajustando ar-condicionado para
    clima de verão")
    AR _ CONDICIONADO(VERAO)
print("Ligando o ar-condicionado") AR _
CONDICIONADO(ON)
```

No exemplo, o ar-condicionado será sempre acionado com a mensagem ao usuário: “ligando o ar-condicionado”; porém, será ajustado para a função verão **somente se a temperatura for maior do que 25°**, avisando o usuário por meio da mensagem: “ambiente quente, ajustando ar-condicionado para clima de verão”.

Estrutura condicional if... else

A estrutura `if... else` (do inglês “se... senão”) é um legado da programação estruturada, pois orienta o fluxo da informação em um sentido “*top-down*”, isto é, de cima para baixo, desde o início até o fim do código.

Esse tipo de estrutura atende à situação em que é necessário escolher uma de duas opções, de acordo com o resultado da comparação.

Veja abaixo a sintaxe da instrução `if... else`:

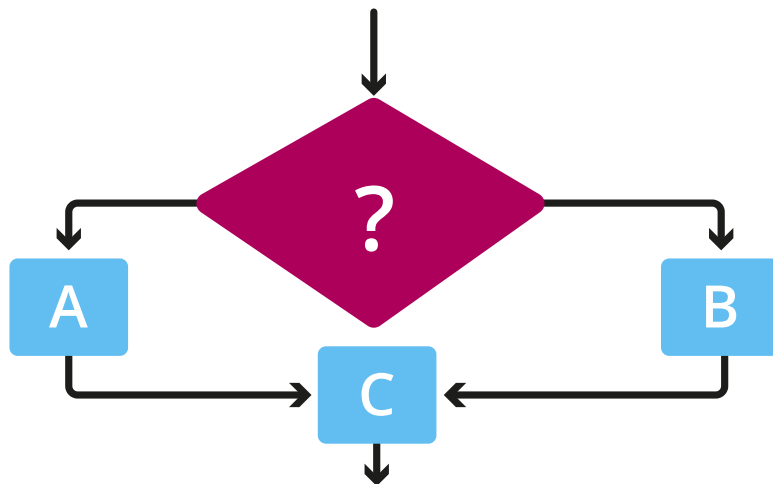
```
if condição lógica: #Bloco A
    Bloco de comandos executados caso a condição lógica seja
    VERDADEIRA
else:
    #Bloco B
    Bloco de comandos executados caso a condição lógica seja
    FALSA
#Bloco C
Comandos executados após o teste.
```

Se a condição lógica for VERDADEIRA, o Bloco A de comandos será executado. No entanto, se a condição lógica for FALSA, somente o Bloco B de instruções será executado.

Após o término da execução das instruções do Bloco A ou do Bloco B, as instruções do Bloco C serão executadas.

A figura a seguir exibe um diagrama com a representação gráfica dessa situação:

Figura 2 - Estrutura condicional if... else



Fonte: o autor.

Vejamos um exemplo:

```
if temperatura>25:
    print("Ambiente quente, ajustando ar-condicionado para
    clima de verão")
    AR _ CONDICIONADO(VERAO)
else :
    print("Ambiente frio, ajustando ar-condicionado para clima
    de inverno")
    AR _ CONDICIONADO(INVERNO)
print("Ligando o ar-condicionado") AR _
CONDICIONADO(ON)
```

No exemplo, o ar-condicionado será sempre acionado, com a mensagem ao usuário “ligando o ar-condicionado”. Será ajustado para o verão, avisando o usuário por meio da mensagem “ambiente quente, ajustando ar-condicionado para clima de verão”, **se a temperatura for maior do que 25°**, ou será ajustado para o inverno, avisando o usuário por meio da mensagem “ambiente frio, ajustando ar-condicionado para clima de inverno”, **se a temperatura for menor ou igual a 25°**.

Estrutura condicional if... elif... else

Essa estrutura permite o encadeamento de vários testes, selecionando conjuntos diferentes de ações, de acordo com o resultado em cascata de cada condição lógica. Em Python, além de proporcionar a geração de um sistema de decisão mais complexo, a estrutura if... elif... else é frequentemente usada no menu de opções para o usuário. Veja abaixo a sintaxe da instrução if... elif... else:

```
if condição lógica1: #Bloco A
    Bloco de comandos executados caso a condição lógica1 seja
    VERDADEIRA
elif condição lógica2: #Bloco B
    Bloco de comandos executados caso a condição lógica1 seja
    FALSA e a condição lógica2 seja VERDADEIRA
elif condição lógica3: #Bloco C
    Bloco de comandos executados caso a condição lógica1 seja
    FALSA, a condição lógica2 seja FALSA e a condição lógica3
    seja VERDADEIRA
    ...
else:
    #Bloco N
    Bloco de comandos executados caso NENHUMA DAS condições
    lógicas sejam VERDADEIRAS

#Bloco X
Comandos executados após o teste.
```

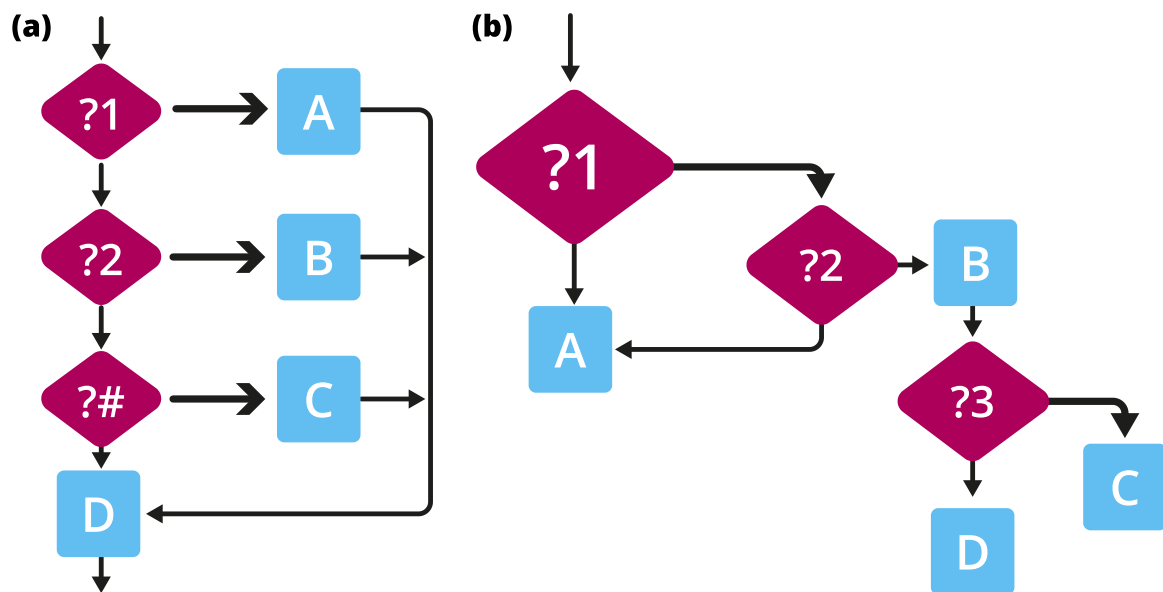
Nessa estrutura, se a condição lógica1 for VERDADEIRA, o Bloco A de comandos será executado. Se, no entanto, a condição lógica1 for FALSA, a condição lógica2 será avaliada e, se for VERDADEIRA, somente o Bloco B de instruções será executado.

Caso as condições lógicas 1 e 2 forem ambas falsas, a condição lógica3 será avaliada e, se for VERDADEIRA, somente o Bloco C de instruções será executado, e assim sucessivamente, para cada novo teste em cadeia.

Por último, se nenhuma condição lógica dos testes em cadeia for VERDADEIRA, então, o Bloco N de instruções será executado. Após o término da execução de algum dos blocos de instruções dos testes do sistema de decisão, as instruções do Bloco X serão executadas.

A figura abaixo exibe um diagrama com a representação gráfica dessa situação:

Figura 3 - Estrutura condicional if... else: (a) funcionamento como menu; (b) funcionamento em cadeia



Fonte: o autor.

Resolução de problemas computacionais utilizando os comandos IF e IF... ELSE



AUTORIA
Cleber Jose Semensate Santos

A partir dos conhecimentos sobre a sintaxe e os usos das diferentes estruturas de decisão do comando `if`, nesta seção, vamos analisar e resolver alguns problemas.

Veja o enunciado a seguir, considerando que o plano diretor de desenvolvimento urbano de uma cidade determina qual é o percentual de área máximo destinado para garagem em relação à área total do terreno da casa, dependendo da localização deste terreno na cidade:

- Para a zona norte da cidade, o percentual máximo é de 25%;
- Para as zonas leste e oeste da cidade, o percentual máximo é de 30%;
- Para a zona sul, menos povoada, o percentual máximo é de 40%.

Uma empresa de arquitetura está com vários contratos e necessita calcular rapidamente esse percentual, antes de iniciar os projetos. Faça um programa que receba as medidas do terreno e da garagem e a zona onde estará localizado o imóvel, calcule o percentual de ocupação da área da garagem em relação ao terreno e emita mensagem sobre o atendimento às regras de ocupação conforme o plano diretor.

Pelo enunciado, existem cinco variáveis, a saber:

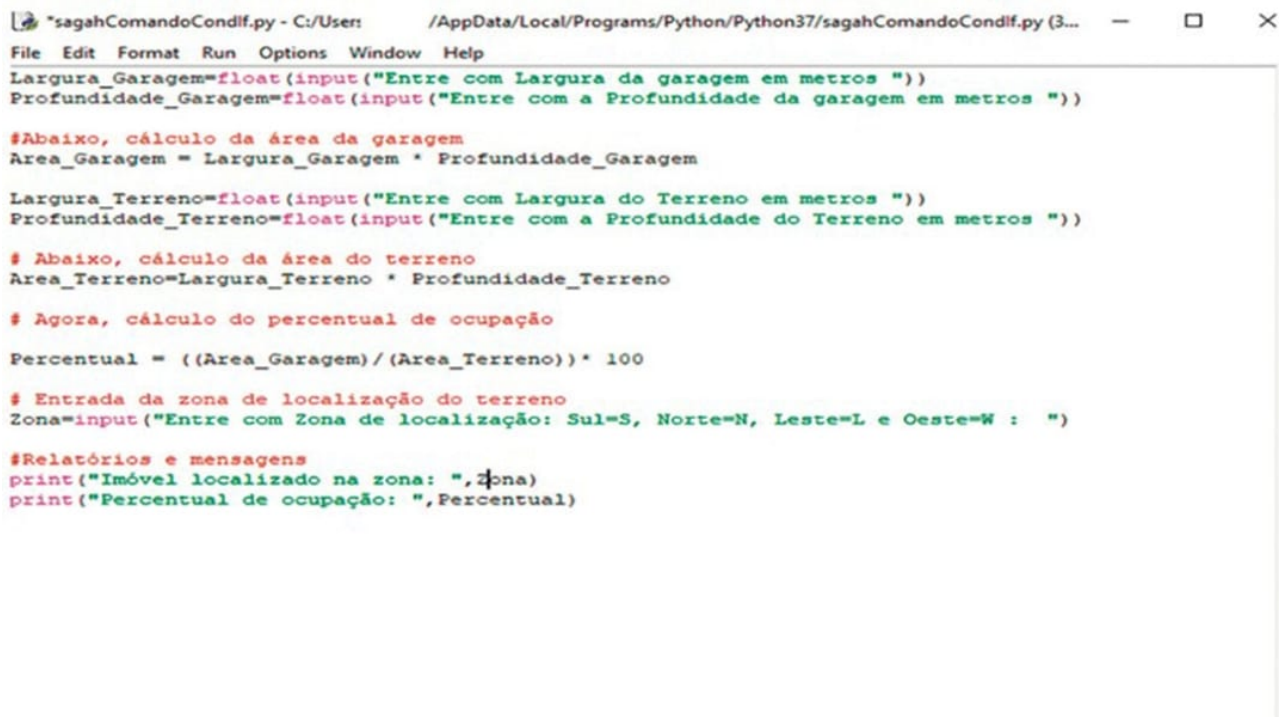
```
largura da garagem em metros  
profundidade da garagem em metros  
largura do terreno em metros  
profundidade do terreno em metros  
zona de localização do terreno
```

Com essas variáveis, deve-se executar as seguintes tarefas:

```
Calcular área do terreno = largura do terreno * profundidade  
do terreno  
Calcular área da garagem = largura da garagem * profundidade  
da garagem  
Calcular percentual de ocupação = (área da garagem / área do  
terreno) * 100  
Apresentar em tela o resultado do percentual de ocupação  
Comparar o percentual com os percentuais definidos por cada  
zona Emitir mensagem sobre a situação – atendimento ou não  
atendi- mento das regras de zoneamento do plano diretor
```

Para tornar claro o código, vamos inicialmente programar a entrada de dados e os cálculos que serão executados, conforme apresentado na figura a seguir:

Figura 4 - Entrada de dados do programa exemplo



```
*sagahComandoCondif.py - C:/Users/ /AppData/Local/Programs/Python/Python37/sagahComandoCondif.py (3...
File Edit Format Run Options Window Help
Largura_Garagem=float(input("Entre com Largura da garagem em metros "))
Profundidade_Garagem=float(input("Entre com a Profundidade da garagem em metros "))

#Abaixo, cálculo da área da garagem
Area_Garagem = Largura_Garagem * Profundidade_Garagem

Largura_Terreno=float(input("Entre com Largura do Terreno em metros "))
Profundidade_Terreno=float(input("Entre com a Profundidade do Terreno em metros "))

# Abaixo, cálculo da área do terreno
Area_Terreno=Largura_Terreno * Profundidade_Terreno

# Agora, cálculo do percentual de ocupação
Percentual = ((Area_Garagem)/(Area_Terreno))* 100

# Entrada da zona de localização do terreno
Zona=input("Entre com Zona de localização: Sul=S, Norte=N, Leste=L e Oeste=W : ")

#Relatórios e mensagens
print("Imóvel localizado na zona: ",Zona)
print("Percentual de ocupação: ",Percentual)
```

Fonte: o autor.

Então, vamos programar a sequência de testes correspondente ao zoneamento, conforme as regras do plano diretor. Observe que, conforme a zona, deve-se testar o percentual de ocupação e comparar com o percentual permitido por zona, emitindo a mensagem de situação de atendimento às normas do plano diretor.

Veja o fragmento de código mostrado na figura a seguir:

Figura 5 - Sequência de testes do programa exemplo

```
if Zona=="N" and Percentual<=25:  
    print("projeto atende norma de zoneamento do plano diretor")  
  
elif Zona=="L" or Zona=="W" and Percentual<=30:  
    print("projeto atende norma de zoneamento do plano diretor")  
  
elif Zona=="S" and Percentual<=40:  
    print("projeto atende norma de zoneamento do plano diretor")  
  
else:  
    print("REVISAR MEDIDAS. projeto NÃO atende norma de zoneamento do plano diretor")
```

Fonte: o autor.

Veja que, nos testes, foram utilizados os operadores lógicos para conectar as condições lógicas referentes à zona e ao percentual de ocupação.

Confira o código completo na figura abaixo:

Figura 6 - Código completo do programa exemplo

```
File Edit Format Run Options Window Help
#Abaixo, cálculo da área da garagem
Area_Garagem = Largura_Garagem * Profundidade_Garagem

Largura_Terreno=float(input("Entre com Largura do Terreno em metros "))
Profundidade_Terreno=float(input("Entre com a Profundidade do Terreno em metros "))

# Abaixo, cálculo da área do terreno
Area_Terreno=Largura_Terreno * Profundidade_Terreno

# Agora, cálculo do percentual de ocupação
Percentual = ((Area_Garagem)/(Area_Terreno))* 100

# Entrada da zona de localização do terreno
Zona=input("Entre com Zona de localização: Sul=S, Norte=N, Leste=L e Oeste=W : ")

#Relatórios e mensagens
print("Imóvel localizado na zona: ",Zona)
print("Percentual de ocupação: ",Percentual)
# Sequencia de decisão

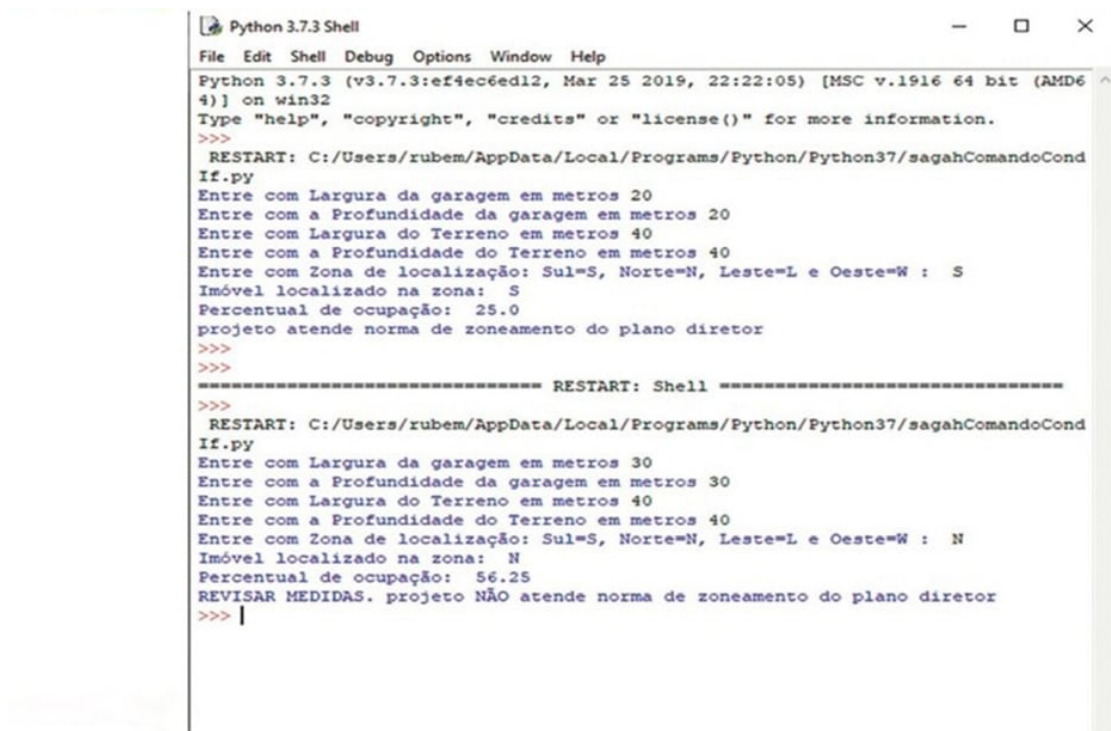
if Zona=="N" and Percentual<=25:
    print("projeto atende norma de zoneamento do plano diretor")
elif Zona=="L" or Zona=="W" and Percentual<=30:
    print("projeto atende norma de zoneamento do plano diretor")
elif Zona=="S" and Percentual<=40:
    print("projeto atende norma de zoneamento do plano diretor")
else:
    print("REVISAR MEDIDAS. projeto NÃO atende norma de zoneamento do plano diretor")
```

Ln: 17 Col: 43

Fonte: o autor.

A próxima figura exibe a execução do programa:

Figura 7 - Execução do programa exemplo mostrado na figura 6 em duas situações



```
Python 3.7.3 Shell
File Edit Shell Debug Options Window Help
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 22:22:05) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/rubem/AppData/Local/Programs/Python/Python37/sagahComandoCond
If.py
Entre com Largura da garagem em metros 20
Entre com a Profundidade da garagem em metros 20
Entre com Largura do Terreno em metros 40
Entre com a Profundidade do Terreno em metros 40
Entre com Zona de localização: Sul=S, Norte=N, Leste=L e Oeste=W : S
Imóvel localizado na zona: S
Percentual de ocupação: 25.0
projeto atende norma de zoneamento do plano diretor
>>>
>>>
===== RESTART: Shell =====
>>>
RESTART: C:/Users/rubem/AppData/Local/Programs/Python/Python37/sagahComandoCond
If.py
Entre com Largura da garagem em metros 30
Entre com a Profundidade da garagem em metros 30
Entre com Largura do Terreno em metros 40
Entre com a Profundidade do Terreno em metros 40
Entre com Zona de localização: Sul=S, Norte=N, Leste=L e Oeste=W : N
Imóvel localizado na zona: N
Percentual de ocupação: 56.25
REVISAR MEDIDAS. projeto NÃO atende norma de zoneamento do plano diretor
>>> |
```

Fonte: o autor.

Existem formas diferentes de utilização da instrução if, combinando estruturas simples e complexas.

Veja, na figura a seguir, solução do problema anterior com nova sequência de decisão, que produz o mesmo resultado:

Figura 8 - Versão alternativa para o segmento de testes do programa exemplo

```
# Nova Sequencia de decisão
if Zona=="N":
    if Percentual<=25:
        print("projeto atende norma de zoneamento do plano diretor")
elif Zona=="L" or Zona=="W":
    if Percentual<=30:
        print("projeto atende norma de zoneamento do plano diretor")
elif Zona=="S":
    if Percentual<=40:
        print("projeto atende norma de zoneamento do plano diretor")
else:
    print("REVISAR MEDIDAS. projeto NÃO atende norma de zoneamento do plano diretor")
```

Fonte: o autor.

Como vimos, as estruturas de programação `if`, `if... else` e `if... elif... else` cobrem todas as formas e os diferentes modos de decisão. Assim, a principal tarefa do programador é a correta definição da lógica necessária para o processo de decisão, bem como das variáveis envolvidas para orientar essa decisão.



SAIBA MAIS

Python possui um amplo conjunto de funções prontas, denominadas pacotes, para uma grande variedade de usos, como criação de jogos, manipulação de dados, geração de gráficos, processamento de imagens, dentre outros projetos.

Saiba mais sobre esses excelentes recursos acessando o link a seguir:

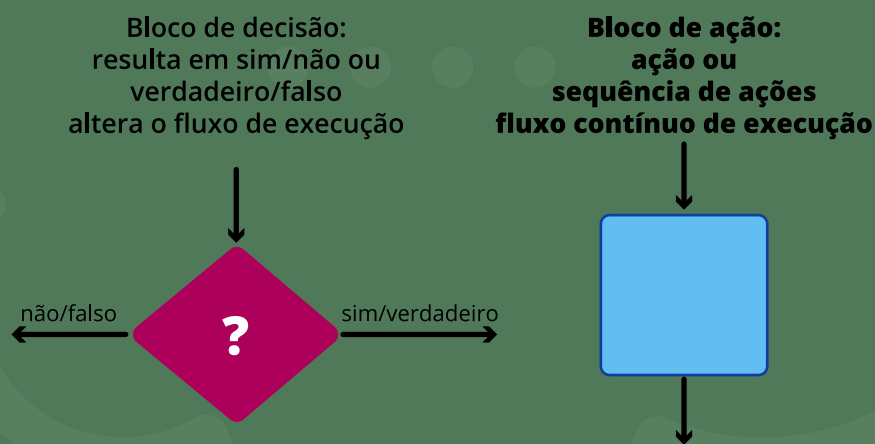
ACESSAR



REFLITA

As representações gráficas de algoritmos e programas são amplamente utilizadas, tornando mais compreensível o comportamento funcional. O fluxograma possui símbolos que representam diferentes tipos de instruções.

Os símbolos mais utilizados, inclusive fora da área de programação (sendo empregados na apresentação de protocolos de operação, protocolos de atendimento, protocolos de emergência, dentre outros), são os seguintes:



Fonte: o autor

Conclusão - Unidade 3

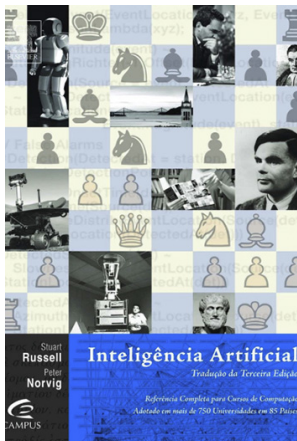
Olá, aluno (a)!

Podemos ver que a lógica de decisão adotada por uma máquina está registrada em seus circuitos de memória, ou seja, tal lógica foi previamente elaborada e programada na máquina. Em outras palavras, um ser humano com inteligência – mais especificamente, um programador – desenvolveu e implantou na máquina o processo de decisão e os procedimentos subsequentes, sendo ele, então, o responsável direto pelas ações da máquina, considerando que máquinas não pensam, não raciocinam, não são inteligentes.

Desse modo, vimos a necessidade de formalizar o processo de decisão que envolve qualquer tarefa a ser realizada pela máquina, o que constitui um importante e necessário conhecimento do programador. Sendo assim, este fará uso operadores lógicos e instruções específicas para implantar suas decisões.

Neste capítulo, você estudou a forma de programar o processo de decisão na máquina, aprendendo sobre as estruturas lógicas que governam esse processo. Você também analisou em detalhes o comando `if`, empregado na programação de computadores, e suas formas de utilização. Por fim, você verificou a forma como resolver problemas utilizando a instrução `if` na linguagem de programação Python.

Parabéns por mais esta etapa e continue firme nos estudos!



Livro

Inteligência Artificial

Autor: Stuart Russell e Peter Norvig

Editora: Elsevier

Sinopse: um assunto que domina o imaginário do ser humano, da ficção científica à realidade de inúmeros estudos sobre o tema, a Inteligência Artificial é apontada como polêmica e inevitável. O que aconteceria se as máquinas fossem capazes de tomar, sozinhas, decisões complexas? Em que patamar está esse avanço hoje? Qual o seu benefício para o futuro? Peter Norvig, diretor do Google, e o professor Stuart Russell, diretor do Center for Intelligent Systems, tratam do tema de forma aprofundada na 3ª edição desta obra apontada como a número um no estudo da Inteligência Artificial. Seu conteúdo, que abrange da introdução à teoria até a prática da aplicação da tecnologia de IA e seus marcos algoritmos como a solução do jogo de damas, é adotado por mais de 600 universidades em 60 países. Compre já o seu livro e acompanhe o pensamento em constante evolução fruto dos últimos cinquenta anos de pesquisas.



Filme

Exact Instructions Challenge - THIS is why my kids hate me.

Ano: 2017

Sinopse: Desenvolver um algoritmo para fazer alguma coisa não é uma tarefa simples. No vídeo disponível no link a seguir, conheça Josh Darnit, que publicou uma série de vídeos no YouTube chamada Exact Instructions Challenge (em português, "O desafio das instruções exatas"). Nessa série de vídeos, Josh desafia seus filhos a criarem uma sequência de instruções para a realização de tarefas simples, como fazer um sanduíche de pasta de amendoim ou um desenho.

Referências

LAW, S. **Filosofia**: guia ilustrado Zahar. 2. ed. Rio de Janeiro: Zahar, 2008.

LUTZ, M. **Programming Python**. 4. ed. Sebastopol: O'Reilly, 2011.

RUSSELL, S.; NORVIG, P. **Inteligência artificial**. 3. ed. Rio de Janeiro: Elsevier, 2013.



| Unidade 4

Comando for, variáveis contadoras e acumuladoras



AUTORIA

Cleber Jose Semensate Santos

Introdução

Olá, caro(a) aluno(a)!

Neste capítulo, você vai estudar o comando `for` da linguagem Python. Com ele, você poderá criar variáveis contadoras e variáveis acumuladoras, que são muito importantes para manipular alguma variável do tipo lista.

Por fim, você vai analisar como funciona o teste de mesa para algoritmos que usam o comando `for`. Esse teste é uma importante ferramenta, que pode ser empregada para testar e validar um algoritmo que foi desenvolvido.

Boa leitura e bons estudos!

O comando for e as variáveis contadoras e acumuladoras

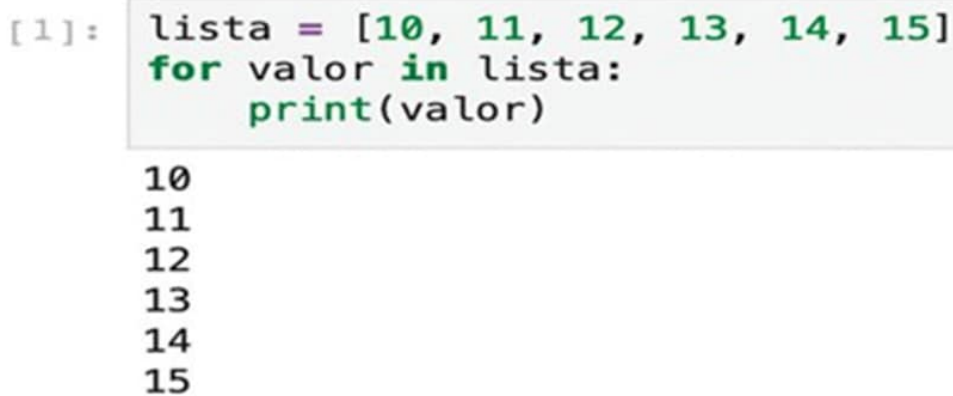


AUTORIA
Cleber Jose Semensate Santos

O comando `for` consiste em uma estrutura de repetição baseada em alguma variável do tipo lista. Com esse comando, é possível executar um código específico para cada elemento da lista. A lista pode ser qualquer tipo específico que você já tenha disponível no seu código.

Na figura a seguir você pode ver um exemplo do uso do comando `for`. Nesse exemplo, foi declarada uma variável do tipo lista, contendo do número 10 ao número 15. O comando `for` faz com que a variável `valor` assuma cada um dos elementos da lista, um de cada vez. Dessa forma, o comando `print` é capaz de imprimir cada um dos números da lista (PYTHON SOFTWARE FOUNDATION, 2019).

Figura 1 - Exemplo de uso do comando `for` para imprimir os numeros de uma lista de valores prévios



```
[1]: lista = [10, 11, 12, 13, 14, 15]
    for valor in lista:
        print(valor)
```

10
11
12
13
14
15

Fonte: o autor.

Quando você não tem uma lista, mas, mesmo assim, quer usar o comando `for`, é possível usá-lo com o comando `range`. Veja a figura 2 para um exemplo dessa abordagem.

O comando `range` cria uma lista de valores com cinco números inteiros, começando em 0. Na próxima figura, você também pode observar outro exemplo de uso do comando `for` junto ao comando `range`. Utiliza-se a expressão `range (4, 8)` para que a lista numérica seja iniciada em 4 e vá até o número 8.

Figura 2 - Exemplo de uso do comando for para imprimir os números de uma lista de valores criada com o comando range

```
[1]: for valor in range(5):  
      print(valor)  
  
0  
1  
2  
3  
4  
  
[1]: for valor in range(4, 8):  
      print(valor)  
  
4  
5  
6  
7
```

Fonte: o autor.

Outros tipos de variáveis também podem ser usados junto com o comando for: podemos usar uma variável de texto, ou *string*, junto com o comando for e, na figura a seguir, temos um exemplo dessa aplicação. Repare que o texto nada mais é do que uma lista de caracteres.

O comando for é capaz de atribuir à variável *caracter* cada letra da variável *texto*, uma de cada vez. Observa-se que o comando for é extremamente útil quando queremos percorrer cada elemento de uma lista (PYTHON SOFTWARE FOUNDATION, 2019).

Figura 3 - Exemplo de uso do comando for para imprimir cada caractere de uma variável do tipo string

```
[1]: texto = "Python"
    for caracter in texto:
        print(caracter)
```

P
y
t
h
o
n

Fonte: o autor.

Variáveis Contadoras e Acumuladoras

É muito comum realizarmos alguma operação numérica com listas. Por exemplo, se quisermos contar quantos caracteres existem em uma frase, podemos usar a função for, juntamente com uma variável, para a contagem de caracteres. Na figura 4, temos um exemplo que ilustra exatamente essa situação. Observe que, para esse exemplo, foi criada a variável qtdeLetras.

Dentro da estrutura do comando for, a cada letra que ele percorre na frase, a variável qtdeLetras é incrementada, isto é, é acrescida uma unidade para seu valor. Pode-se dizer que essa variável é uma **variável contadora**. Repare que a variável contadora não considera o valor da variável de interação do comando for, a qual, nesse exemplo, é a variável letra.

Figura 4 - Exemplo do uso do comando for para fazer a contagem de caracteres de uma frase

```
[1]: frase = "A ligeira raposa marrom ataca o cão preguiçoso"
qtdeLetras = 0
for letra in frase:
    qtdeLetras += 1
print("A frase possui " + str(qtdeLetras) + " letras.")
A frase possui 47 letras.
```

Fonte: o autor.

Às vezes, precisamos considerar o valor da variável de interação do comando for. Um exemplo clássico é o algoritmo para o cálculo de uma média, que está ilustrado na figura 5.

Observe que a variável soma vai aumentando seu valor a cada interação do comando for, sempre se somando com o valor da variável de interação do comando for. A variável soma é uma **variável acumuladora**, pois ela está, a cada interação, acumulando o seu valor com o valor da variável de interação do comando for.

Figura 5 - Exemplo do uso do comando for para fazer o cálculo da média de uma lista de valores numéricos

```
[1]: dados = [[1, 2, 3],[4, 5, 6],[7, 8, 9]]

soma = 0
qtde = 0

for valor in dados:
    soma += valor
    qtde += 1

print(soma)
print(qtde)

media = soma / qtde

print(media)

46
6
4.833333333333333
```

Fonte: o autor.

Apesar de seu uso simples, as variáveis contadoras e as variáveis acumuladoras podem promover vários erros no algoritmo. A fim de evitar que esses erros ocorram, é necessário sempre fazer o **teste de mesa**.

Testes de mesa com estrutura de repetição for



AUTORIA
Cleber Jose Semensate Santos

Durante o desenvolvimento dos algoritmos, é necessário que sejam feitos testes, a fim de certificar que o funcionamento corresponde ao esperado. O teste que é realizado durante a fase de desenvolvimento do algoritmo é conhecido como **teste de mesa**, conforme pontua Schach (2009). Ele é realizado geralmente à mão, para cada interação do comando `for`. É muito importante observar o valor atribuído a cada variável durante cada interação. dessa forma, é possível acompanhar a evolução da variável sob análise.

Teste De Mesa Para Uma Variável Acumuladora

As variáveis acumuladoras apresentam uma alteração do seu valor a cada interação do comando `for`. Dessa forma, quando for realizado um teste de mesa que envolva uma variável acumuladora, é imperativo que seja tomado nota de seu valor a cada interação. Caso o algoritmo apresente alguma falha, por meio da análise da evolução da variável acumuladora, será possível determinar em qual local do código se encontra a falha de programação.

Na figura 6, temos um algoritmo simples que soma os números pares até 10. Como a condição de paridade de um número é que ele seja divisível por 2, esse algoritmo deve ainda fazer essa verificação. Ele a faz por meio do comando `i f`. Lembrando que o operador `%` retorna o resto inteiro da divisão de dois números. Caso o número seja par (divisível por 2), ele será somado na variável acumuladora `soma`. Para verificar se esse algoritmo está funcionando como esperado, vamos realizar um teste de mesa para ele.

Figura 6 – Algoritmo para somar os números pares ate 10

```
[1]: soma = 0

for numero in range(10):
    if (numero % 2) == 0:
        soma += numero
```

Fonte: o autor.

No quadro a seguir, está registrado o valor de cada variável em cada interação. Observe que a variável soma só é alterada quando a variável número assume um valor par.

Quadro 1 - Valores para o teste de mesa do algoritmo da figura

Interação	Variável número	Variável soma
1	0	0
2	1	0
3	2	2
4	3	2
5	4	6
6	5	6
7	6	12
8	7	12
9	8	20
10	9	20

Fonte: o autor.

Teste De Mesa Com Variáveis Contadoras

O teste de mesa envolvendo variáveis contadoras é um pouco mais simples do que com variáveis acumuladoras. O valor das variáveis contadoras está diretamente associado ao número de interações que o laço `for` realiza.

Um cuidado especial deve ocorrer quando a variável contadora estiver associada a um comando condicional `if`. Nesse caso, a análise da condição deve ser feita com bastante atenção, porque o valor da variável contadora está diretamente associado a essa condição.

Na figura 7, temos um algoritmo capaz de contar quantos números múltiplos inteiros de 3 existem nos 20 primeiros números naturais. Para isso, foram utilizadas duas variáveis contadoras: `noVerificados` e `noMúltiplos`.

Na variável `noVerificados`, deverá ser feita a contagem para determinar quantos números foram verificados durante todo o algoritmo. Repare que essa variável não é condicionada. Já a variável `noMultiplos` será incrementada apenas se a condição do comando `if` for satisfeita; assim, essa variável vai contar o número de múltiplos de 3 que existem dentro do intervalo de análise do algoritmo.

Figura 7 - Algoritmo para contar quantos múltiplos inteiros de 3 existem nos 20 primeiros números naturais

```
[1]: noVerificados = 0
    noMultiplos = 0

    for numero in range(20):
        noVerificados += 1
        if (numero % 3) == 0:
            noMultiplos += 1

    print(noVerificados)
    print(noMultiplos)
```

20
7

Fonte: o autor.

Para fazer o teste de mesa do algoritmo da figura 7, foi usada a tabela do Quadro 2 como apoio. Conforme o comando percorria o intervalo de valores definidos pelo comando `range(20)`, ambas as variáveis tiveram seu valor anotado. Os resultados obtidos pela execução do algoritmo, conforme já ilustrado na figura 7, são idênticos ao resultado do teste de mesa, que está relacionado no Quadro 2.

Quadro 2 - Valores para o teste de mesa do algoritmo da figura 7

Número	noVeri- ficados	noMul- tiplos	Número	noVeri- ficados	noMul- tiplos
0	1	1	10	11	4
1	2	1	11	12	4
2	3	1	12	13	5
3	4	2	13	14	5
4	5	2	14	15	5
5	6	2	15	16	6
6	7	3	16	17	6
7	8	3	17	18	6
8	9	3	18	19	7
9	10	4	19	20	7

Fonte: o autor.

Aplicações Práticas das variáveis contadoras e acumuladoras



AUTORIA

Cleber Jose Semensate Santos

O comando `for` é uma das estruturas mais básicas existentes na linguagem Python. Porém, ao mesmo tempo em que ela é simples, é muito poderosa. Com esse comando e com o auxílio das variáveis contadoras e das variáveis acumuladoras, é possível resolver vários problemas computacionais, conforme expõe Santos (2018).

Cálculo Do Fatorial De Um Número

O cálculo do fatorial de um número qualquer consiste em uma operação matemática simples. Diz-se que o fatorial de um número é igual ao produto de todos os números decrescentes do número até 0, isto é, o fatorial $n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot (n - k)$, sendo que $k = n - 1$. Por exemplo, o fatorial de 3 é $3! = 3 \cdot 2 \cdot 1 = 6$.

Para implementar esse processo na linguagem Python, por meio do comando `for`, pode ser utilizado o código ilustrado da próxima figura.

Figura 8 - Exemplo do uso do comando `for` para fazer o cálculo do fatorial de um número qualquer

```
[1]: numero = int(input("Digite o número que se deseja determinar o fatorial: "))

fatorial = 1
for termo in range(1, (numero + 1)):
    fatorial *= termo

print("O fatorial de " + str(numero) + "! é : " + str(fatorial))

Digite o número que se deseja determinar o fatorial: 3
O fatorial de 3! é : 6
```

Fonte: o autor.

No exemplo da figura acima, o comando `range` teve seus parâmetros alterados para que a contagem não incluísse o número 0. Repare no exemplo da figura 2, em que o comando `range` inclui o 0, caso não seja informado explicitamente o início e o fim da faixa de valores.

Para determinar o valor do fatorial, foi empregado o operador de atribuição composto `*=`, para fazer o produto da variável fatorial com cada termo, até chegar no número que o usuário escolheu por meio do comando `input`. O comando `int` serve apenas para converter o valor digitado pelo usuário para um número inteiro.

Uma lista numérica pode ser somada à outra de igual dimensão, por meio da soma de cada elemento individual. Por exemplo, se tivermos uma lista $A = [2, 3, 4]$ e quisermos somar com outra lista $B = [7, -3, 2]$, basta fazer a soma elemento a elemento. Dessa forma, a soma dessas duas listas é dada por: $A + B = [(2 + 7), (3 + (-3)), (4 + 2)] = [9, 0, 6]$, conforme apontam Forbellone e Eberspächer (2005).

Para fazer essa mesma operação usando a linguagem Python, podemos usar o comando `for` para fazer o controle iterativo, elemento a elemento.

Na figura 9, está ilustrado esse mesmo exemplo, porém, em linguagem Python. Observe que foi utilizada uma variável auxiliar, denominada `índice`, juntamente com o comando `range`.

O comando `range`, nesse caso, cria uma lista de números inteiros iniciada em 0, com um total de três elementos. A variável `índice` é usada, nesse caso, para representar o índice de cada lista, que será obtido para fazer a soma. Por exemplo, quando `índice = 0`, o algoritmo estará obtendo o elemento 0 da lista A e o elemento 0 da lista B . Por fim, a lista C será alimentada com o resultado dessa operação.

Figura 9 - Exemplo do uso do comando `for` para fazer a soma de duas listas de tamanhos iguais

```
[1]: A = [2, 3, 4]
      B = [7, -3, 2]

      C = []

      for indice in range(3):
          C.append(A[indice] + B[indice])

      print(C)

      [9, 0, 6]
```

Encontrando Números Primos

Número primo é todo aquele que é divisível apenas por 1 e por si mesmo. Por exemplo, o número 7 é primo, porque só conseguimos dividir 7 por 7 e por 1 para obter um resultado inteiro ou uma divisão sem resto.

Na computação, os números primos são muito importantes, porque, a partir deles, são construídos vários algoritmos de criptografia. Quando o número é pequeno, é fácil descobrir se ele é ou não um número primo; porém, quanto maior o número fica, mais difícil é descobrir se ele é ou não primo, conforme destacam Holanda Filho e Gomes (2019).

É possível usar o comando `for` para criar um algoritmo que busque os números primos dentro de um intervalo de valores. Na figura 10, está ilustrado um algoritmo que usa dois comandos `for` para essa tarefa. O primeiro comando `for` percorre uma lista de 20 números iniciada em 0, que é produzida pelo comando `range`. Assim, esse algoritmo vai verificar quais são os números primos que existem até o número 20. O segundo comando `for` faz o teste de divisão com todos os números de 1 até o próprio número, para verificar se existe ou não resto na divisão.

Observe que foi utilizado o comando `range`, percorrendo de 1 até o número + 1. Isso é necessário porque o comando `range` cria uma lista iniciada em 0, e não é possível dividir nenhum número por 0. Em seguida, a variável `div` funciona como uma variável contadora, armazenando a quantidade de números que conseguiram dividir o número sob teste, resultando em um resto 0 (o operador `%` retorna o resto inteiro da divisão). Caso o número de divisores seja igual a 2, o número pode ser considerado primo.

Figura 10 - Exemplo do uso do comando for para determinar números primos

```
[1]: numPrimos = []

    for numero in range(20):
        div = 0
        for divisor in range(1, numero + 1):
            if (numero % divisor) == 0:
                div += 1
        if div == 2:
            numPrimos.append(numero)

    print(numPrimos)

[2, 3, 5, 7, 11, 13, 17, 19]
```

Fonte: o autor.



SAIBA MAIS

No link a seguir, você poderá verificar porque os números primos são tão importantes para a criptografia:

ACESSAR



REFLITA

Além de utilizarmos o `for` (enquanto) para replicarmos um bloco de código, também utilizamos este comando para iterarmos uma sequência — lista, tupla etc. — de forma muito mais simples do que o comando `While`.

Fonte: Santos (2018).

Conclusão - Unidade 4

Prezado(a) aluno(a),

Neste capítulo, você estudou o comando for da linguagem Python. Com ele, você conseguiu criar variáveis contadoras e variáveis acumuladoras, que são muito importantes para manipular alguma variável do tipo lista.

Além disso, você analisou o funcionamento do teste de mesa para algoritmos que usam o comando for. É importante ressaltar que esse teste é uma importante ferramenta a ser empregada para testar e validar algoritmo que foram desenvolvidos.

Sendo assim, foi adquirido o conhecimento necessário para a disciplina de algoritmos e lógica de programação!

Bons estudos e continue firme na jornada!

Material Complementar



Livro

Engenharia de Software: Os Paradigmas Clássico e Orientado a Objetos

Autor: Stephen R. Schach

Editora: McGraw-Hill

Sinopse: este livro apresenta uma excelente introdução aos fundamentos da engenharia de software. Com uma abordagem equilibrada dos fundamentos teóricos da engenharia de software, assim como dos aspectos mais práticos do ciclo de vida do software, o livro trata tanto das técnicas tradicionais como das técnicas orientadas a objetos. Em um estilo simples e fácil de ser compreendido pelos estudantes, conceitos complexos são apresentados de forma clara e compreensível. O livro inclui também processos ágeis de software com código-fonte aberto e também o Processo Unificado, extremamente importante no desenvolvimento de software orientado a objetos.



Filme

Metodologia Ágil de Desenvolvimento de Software

Ano: 2012

Sinopse: o curso de Extreme Programming - metodologia ágil de desenvolvimento de software, por meio da Educação a Distância, oferece ao profissional gerente de TI, desenvolvedores, programadores a metodologia XP e muito mais.

Referências

FORBELLONE, A. L. V.; EBERSPÄCHER, H. F. **Lógica de programação:** a construção de algoritmos e estruturas de dados. 3. ed. São Paulo: Pearson Prentice Hall, 2005.

HOLANDA FILHO, I. O.; GOMES, R. L. R. **Criptografia e números primos:** um namoro que deu certo. s.d. Disponível em: <https://www.portaleducacao.com.br/conteudo/artigos/pedagogia/criptografia-e-numeros-primos-um-namoro-que-deu-certo/56347>. Acesso em: 2 abr. 2020.

PYTHON SOFTWARE FOUNDATION. **The Python tutorial.** Wilmington, 2019. Disponível em: <https://docs.python.org/3/tutorial/index.html>. Acesso em: 9 jun. 2019.

SANTOS, M. G. **Algoritmos e programação.** Porto Alegre, 2018.

SCHACH, S. R. **Engenharia de software:** os paradigmas clássico & orientado a objetos. 7. ed. São Paulo, McGraw-Hill, 2009.

Considerações Finais

Caro(a) aluno(a), assim terminamos nossa jornada!

Foram quatro unidades que passearam por diversos temas importantes, os quais giram em torno de Algoritmos e Lógica de Programação, além da importância de dominar a programação. Esta introduziu uma linguagem de programação específica, o Python, com o objetivo de auxiliá-lo a desenvolver seus próprios programas. Foram abordados os conceitos básicos dessa linguagem, de modo que você conseguiu utilizar o mesmo conceito apresentado em outras linguagens de programação.

No primeiro capítulo da obra, apresentamos os conceitos básicos dos computadores para explicar como os softwares funcionam. Dessa forma, você conseguiu entender a importância da programação. Além disso, este capítulo explicou como configurar seu computador para desenvolver os programas.

No capítulo seguinte, foi introduzido um conceito básico de linguagem de programação, a variável, que é um espaço de memória onde podemos guardar informação. Nesse mesmo capítulo, foi observado que, com as variáveis, também são realizadas operações.

No terceiro capítulo, o conceito de condições foi apresentado. Os programas executaram códigos capazes de realizar decisões e modificar sua execução de acordo com as condições em que o programa se encontra.

No quarto capítulo foi trabalhado o conceito de laço de repetição ou, em inglês, loops, nos quais partes de código podem ser copiados, possibilitando a realização de tarefas repetidas.

Uma das características importantes dos programas é interagir com os usuários. Nesse sentido, foi apresentado também como você pode mostrar mensagens e receber entrada do usuário.

Uma das vantagens de programas e código é você poder reutilizá-los com o uso de funções, como mostrado nos tópicos de funções. Mostramos, assim, os princípios básicos da programação para que você possa desenvolver programas.

Não pare por aqui! Vamos em frente!

Um abraço!